INDUSTRIAL INTELLIGENCE AGENT: A MULTI-TENANT PLATFORM FOR MACHINE ANOMALY DETECTION AND EVIDENCE-BACKED MAINTENANCE INVESTIGATION

ABSTRACT

Factory operations teams receive a continuous flood of machine telemetry, but correlating sensor behaviour with maintenance history and equipment documentation fast enough to act before downtime occurs remains a manual task. This project presents the **Industrial Intelligence Agent (IIA)**, a multi-tenant decision-support platform that ingests machine sensor data, detects anomalies, ranks machines by health risk, and answers operational questions such as 'why is M-04 underperforming?' with an evidence-backed investigation report.

The system is implemented in Python as a layered application. A FastAPI service exposes tenant-scoped REST endpoints; a Streamlit dashboard consumes only that API; PostgreSQL 16 stores readings, hourly summaries, anomalies, maintenance records, jobs, users and audit trails, with row-level security (RLS) enforcing tenant isolation for two least-privilege database roles.[The application has a FastAPI backend that handles requests and manages data for different companies separately. Users interact with the application through a Streamlit dashboard, which communicates with FastAPI to get or submit information. The application uses PostgreSQL 16 as its database to store machine sensor readings, reports, detected problems, maintenance details, background tasks, user information, and activity records. It also uses Row-Level Security (RLS) to ensure that one company cannot access another company's data. Finally, it uses two database user roles with limited permissions, so each role can only access the data and perform the operations it is authorized to do.] A background worker, using a PostgreSQL job queue (`FOR UPDATE SKIP LOCKED`), validates tenant CSV uploads against a declarative column and unit mapping, loads long-format readings idempotently, and runs a PySpark anomaly-detection pass (rolling z-score and registered-range rules) in local mode. A scheduled pull connector for Google Cloud Storage feeds the same pipeline. The investigation agent is a LangGraph state graph in which sensor and anomaly evidence, risk assessment and root-cause candidates are deterministic; an optional Gemini-based planner may decide only whether to pull maintenance history and search a 62-chunk maintenance knowledge base (BM25 locally, or Vertex AI embeddings with Pinecone), and an optional Gemini synthesis step writes the narrative under a guardrail that rejects numbers absent from the evidence.

The repository contains 34 test modules with 430 test functions covering tenant isolation, RLS, authentication, the agent, retrieval, ingestion, connectors and migrations. In preparing this report, a subset of seven offline-capable test modules was executed (152 passed, 6 skipped, 0 failed); the remaining results quoted are those recorded in the project's own change logs and are labelled as such. The platform is

demonstrated on synthetic and public data only (the UCI AI4I 2020 dataset plus a generated 18-machine fleet); it has not been deployed at a real factory, and its risk thresholds are calibrated on synthetic data.

Keywords: predictive maintenance, anomaly detection, multi-tenancy, row-level security, LangGraph, retrieval-augmented generation, PySpark, FastAPI.

1. A "Multi-tenant decision-support platform" is a software system that allows multiple organizations or companies to use the same application while keeping their data, users, and operations separate. It helps them make better decisions using data, analytics, and AI.
2. A layered application is a software architecture in which the application is divided into separate logical layers, such as presentation, API, business logic, data access, and database layers. Each layer has a specific responsibility, making the application easier to maintain, test, and scale.
3. LangGraph State Graph is a way to build AI applications where different tasks are connected in a specific workflow, and the application remembers information (called state) while moving between those tasks.
4. A Gemini-based planner is an AI component that uses Google's Gemini model to understand a user's request, think about what needs to be done, and create a step-by-step plan to solve the problem.
5. A Gemini synthesis step is the stage where Google's Gemini AI model collects information gathered from different sources or previous workflow steps and combines it into one clear, meaningful answer.

TABLE OF CONTENTS

CHAPTER / SECTION

TITLE

1. INTRODUCTION 1

1.1 BACKGROUND AND CONTEXT 1
1.2 OVERVIEW OF THE APPLICATION DOMAIN 1
1.3 INTRODUCTION TO THE PROJECT 1
1.4 NEED FOR THE PROPOSED SYSTEM 2
1.5 SCOPE AND SIGNIFICANCE 2
1.6 ORGANISATION OF THE REPORT 2

2. LITERATURE REVIEW AND RELATED TECHNOLOGIES 4

2.1 MAINTENANCE STRATEGIES AND CONDITION MONITORING 4
2.2 ANOMALY DETECTION IN SENSOR DATA 4
2.3 PUBLIC DATA AND FAILURE MODES 4
2.4 DISTRIBUTED DATA PROCESSING 5
2.5 RETRIEVAL-AUGMENTED GENERATION AND LEXICAL RETRIEVAL 5
2.6 LLM AGENTS, TOOL USE AND HALLUCINATION 5
2.7 MULTI-TENANT ISOLATION AND ACCESS CONTROL 5
2.8 COMPARISON OF APPROACHES 6
2.9 RESEARCH AND IMPLEMENTATION GAP 6

3. MOTIVATION, PROBLEM STATEMENT AND OBJECTIVES 7

3.1 MOTIVATION 7
3.2 PROBLEM STATEMENT 7
3.3 EXISTING SYSTEM AND ITS LIMITATIONS 7
3.4 PROPOSED SYSTEM 8
3.5 OBJECTIVES 8
3.5.1 Data ingestion and detection 8
3.5.2 Agent and retrieval 8
3.5.3 Security, operations and quality 8
3.6 SCOPE AND CONSTRAINTS 8
3.7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS 9

4. SYSTEM ANALYSIS AND DESIGN 11

4.1 OVERALL SYSTEM ARCHITECTURE 11
4.2 MAJOR MODULES AND RESPONSIBILITIES 11
4.3 COMPONENT INTERACTION 12
4.4 USE-CASE VIEW 13
4.5 DATA-FLOW DIAGRAM 14
4.6 SEQUENCE DIAGRAMS 14
4.6.1 Investigation request 14
4.6.2 Upload, mapping and background job 15
4.7 DATABASE DESIGN 15
4.8 API AND SERVICE ARCHITECTURE 17
4.9 SECURITY AND AUTHORIZATION DESIGN 18
4.10 RELEVANT ALGORITHMS AND WORKFLOW LOGIC 20
4.10.1 Anomaly detection 20

4.10.2 Deterministic risk assessment 20
4.10.3 Agent workflow 21
4.10.4 Queue claim and retry 22
5. METHODOLOGY AND IMPLEMENTATION 24
5.1 DEVELOPMENT METHODOLOGY 24
5.2 TECHNOLOGY STACK AND JUSTIFICATION 24
5.3 ENVIRONMENT AND CONFIGURATION 25
5.4 BACKEND IMPLEMENTATION 26
5.5 DATA PIPELINE AND ANOMALY DETECTION (PYSPARK) 26
5.6 TENANT ONBOARDING AND MAPPING 26
5.7 DATABASE IMPLEMENTATION 27
5.8 AUTHENTICATION AND AUTHORIZATION IMPLEMENTATION 28
5.9 AI AND AGENT IMPLEMENTATION 28
5.9.1 What is conventional logic and what is model-driven 28
5.9.2 Graph construction and tracing 29
5.9.3 Synthesis guardrail 29
5.10 RETRIEVAL IMPLEMENTATION 29
5.11 BACKGROUND JOBS AND CONNECTORS 30
5.12 FRONTEND IMPLEMENTATION 31
5.13 DOCKERIZATION AND DEPLOYMENT ARCHITECTURE 31
5.14 END-TO-END EXECUTION WORKFLOW 32
6. TESTING, RESULTS AND DISCUSSION 33
6.1 TESTING METHODOLOGY 33
6.2 TEST ENVIRONMENT AND PREREQUISITES 33
6.3 TEST INVENTORY 33
6.4 TEST CASES: EXPECTED BEHAVIOUR 34
6.5 RESULTS 35
6.5.1 Tests executed while preparing this report [Run here] 35
6.5.2 Results recorded in the project documentation [Reported] 35
6.6 APPLICATION SCREENSHOTS 36
6.7 DISCUSSION OF OBSERVED BEHAVIOUR 37
6.8 LIMITATIONS, EDGE CASES AND KNOWN ISSUES 37
6.9 DISCREPANCIES BETWEEN DOCUMENTATION AND SOURCE 38
7. CONCLUSION AND FUTURE ENHANCEMENTS 39
7.1 SUMMARY OF THE COMPLETED WORK 39
7.2 MAJOR TECHNICAL CONTRIBUTIONS 39
7.3 EXTENT TO WHICH THE OBJECTIVES WERE ACHIEVED 39
7.4 CURRENT LIMITATIONS 40
7.5 REALISTIC FUTURE ENHANCEMENTS 40
8. REFERENCES 41
APPENDICES 43

LIST OF FIGURES

Figure 4.1: System architecture of the Industrial Intelligence Agent 11
Figure 4.2: Use cases by actor 13
Figure 4.3: Data flow from raw export to investigation report 14
Figure 4.4: Sequence of a `POST /investigate` request 14
Figure 4.5: Sequence of upload, mapping confirmation and job execution 15
Figure 4.6: Entity-relationship diagram (principal tables and keys) 16
Figure 4.7: Authentication and authorization workflow 19
Figure 4.8: Agent workflow, stage 1: target resolution and evidence gathering 21
Figure 4.9: Agent workflow, stage 2: reasoning and report construction 22
Figure 5.1: Docker Compose deployment architecture 31
Figure 6.1: Fleet health page 36
Figure 6.2: Investigations page showing a machine report with citations 36
Figure 6.3: Jobs page showing a succeeded ingest job 37

LIST OF TABLES

Table 2.1: Comparison of conventional approaches with the proposed system 6
Table 3.1: Functional requirements (derived from the implemented API and modules) 9
Table 3.2: Non-functional requirements 9
Table 4.1: Major modules (directory-level, from the source tree) 11
Table 4.2: Role of the main tables 16
Table 4.3: Principal API endpoints and required permissions 17
Table 4.4: Role matrix (from `ROLE_MATRIX`) 17
Table 5.1: Technology stack (versions from `requirements*.txt`, Dockerfiles and compose) 24
Table 5.2: Configuration groups (names only; values deliberately omitted) 25
Table 5.3: Conventional logic versus AI-driven behaviour 28
Table 5.4: Compose services 31
Table 6.1: Test functions by area [Static: counted `def test_` in `tests/`; parametrised tests expand to more cases] 33
Table 6.2: Representative test cases (names from the suite; expected behaviour from the test names and docstrings) 34
Table 6.3: Offline test modules executed for this report [Run here] 35
Table 6.4: Results quoted from `README.md` and `CHANGES_PHASE*.md` (not re-executed) 35
Table 6.5: Known limitations (from README 'Limitations' and code inspection) 37
Table 6.6: Discrepancies found while preparing this report 38
Table 7.1: Objectives against evidence 39
Table 8.1: Status of major capabilities (README 'What's real vs stubbed', checked against the code tree) 44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
AI4I	AI for Industry (UCI AI4I 2020 Predictive Maintenance dataset)
API	Application Programming Interface
BM25	Best Matching 25 (lexical ranking function)
CI	Continuous Integration
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
DFD	Data-Flow Diagram
ER	Entity-Relationship
GCS	Google Cloud Storage
HDF / OSF / PWF / TWF / RNF	Heat dissipation / Overstrain / Power / Tool wear / Random failure modes (AI4I)
JWT	JSON Web Token
LLM	Large Language Model
MCA	Master of Computer Applications
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
RLS	Row-Level Security
SOP	Standard Operating Procedure
SQL	Structured Query Language
UTC	Coordinated Universal Time
UUID	Universally Unique Identifier

CHAPTER 01
INTRODUCTION

1.1 BACKGROUND AND CONTEXT

Modern manufacturing plants are instrumented with sensors that report temperatures, rotational speeds, torque, vibration, energy use and production counters at intervals of seconds to minutes. Maintenance has consequently moved from run-to-failure and fixed-interval servicing towards condition-based and predictive strategies, in which machine condition data decide when an inspection or part replacement is justified [21], [22]. The difficulty is rarely data availability. It is the time and expertise needed to turn a large volume of readings into a defensible statement such as 'this machine has shown a sustained abnormal pattern for the last 24 hours, here is what changed, and here is the documented procedure to follow'.

Large language models (LLMs) make it tempting to answer such questions conversationally, but an unconstrained model can state plausible figures that do not exist in the data [8]. In an operations setting a fabricated statistic or an invented risk level is more harmful than no answer. This project therefore takes the opposite design stance: evidence is gathered and risk is computed by deterministic code, and a language model, where one is used at all, is confined to choosing supplementary evidence and to explaining results that were already computed.

1.2 OVERVIEW OF THE APPLICATION DOMAIN

The application domain is industrial condition monitoring and maintenance decision support. Typical inputs are time-stamped sensor readings per machine, a log of maintenance events, and unstructured equipment documentation (manuals, standard operating procedures, safety procedures and incident reports). Typical outputs are machine health status, a ranked list of machines that need inspection, and an explanation linking observed behaviour to candidate causes and to documented actions. Because such a system is naturally offered to several companies, the domain also brings requirements on data separation between customers, access control, auditability and repeatable ingestion of customer-specific file formats.

1.3 INTRODUCTION TO THE PROJECT

The **Industrial Intelligence Agent (IIA)** is a multi-tenant web platform with the following capabilities, each of which is verified against the source code in later chapters:

- **Ingestion** of tenant sensor data from CSV, either uploaded through the API or pulled on a schedule from a Google Cloud Storage bucket, using a declarative mapping of columns, units, normal ranges and thresholds.
- **Anomaly detection** over the stored readings with a PySpark job that combines rolling-window z-score rules and registered normal-range rules, aggregated into hourly health summaries and per-reading anomaly events.
- **Investigation** of a named machine or of the whole fleet through a LangGraph agent that returns a structured report: risk level, root-cause candidates, supporting evidence and citations to maintenance documents.
- **Multi-tenant security**: tenant identity derived from credentials only, PostgreSQL row-level security, role-based permissions (admin, viewer, API key), argon2id password hashing, JWT login, rate limiting and audit logging.
- **A dashboard** (Streamlit) for fleet health, investigations, uploads, jobs and administration, and a containerised deployment with Docker Compose.

1.4 NEED FOR THE PROPOSED SYSTEM

Three needs motivate the system. First, the evidence required for a maintenance decision is spread over sensor trends, historical records and documents, and collecting it by hand is slow. Second, naive LLM assistants are unsuitable for this decision because their numbers and risk labels are not auditable. Third, a platform that serves more than one company must guarantee that one tenant can never read another's machines, readings or documents, and must accept each company's own column names, units and sensor vocabulary without code changes. The project addresses these needs together rather than as separate prototypes.

1.5 SCOPE AND SIGNIFICANCE

The scope of the work reported here is the complete software platform in the submitted repository: data ingestion, detection, storage, agent, retrieval, API, dashboard, security model and deployment configuration. The platform is demonstrated on public and synthetic data. It is not a system deployed at a real factory, and no business-impact figure in the project documentation is a measured outcome. The significance of the work lies in showing how an auditable, tenant-isolated, partly LLM-assisted decision-support pipeline can be assembled from standard open components, with explicit boundaries between deterministic computation and model-generated text.

1.6 ORGANISATION OF THE REPORT

Chapter 2 reviews related technologies and the gap addressed. Chapter 3 states the motivation, problem, objectives and requirements. Chapter 4 presents the analysis and design,

including architecture, data flow, database design and security design. Chapter 5 describes the implementation. Chapter 6 reports testing and the evidence available for results. Chapter 7 concludes and lists future enhancements, and Chapter 8 lists references. Appendices give installation steps, configuration, API examples and a verification log.

CHAPTER 02
LITERATURE REVIEW AND RELATED TECHNOLOGIES

Note on sources. Statements in this chapter that carry a bracketed number are drawn from the external references listed in Chapter 8. Statements about what this project does are findings from the submitted source code and are identified as such. The bibliographic entries were prepared from the author's knowledge of these works and should be checked against the library copies before submission.

2.1 MAINTENANCE STRATEGIES AND CONDITION MONITORING

Jardine, Lin and Banjevic reviewed condition-based maintenance (CBM) and distinguished diagnostics, which identify a fault after it has occurred, from prognostics, which estimate degradation before failure [21]. Lee, Bagheri and Kao described a cyber-physical systems architecture for Industry 4.0 manufacturing in which data from connected machines feed analytics and decision layers [20]. Carvalho et al. systematically reviewed machine-learning methods for predictive maintenance and observed that studies differ widely in data sets, objectives and evaluation, which makes direct comparison of methods difficult [22]. A recurring practical conclusion in this literature is that data quality, labelled failure data and integration with maintenance practice, not algorithm choice alone, determine whether a solution is usable.

2.2 ANOMALY DETECTION IN SENSOR DATA

Chandola, Banerjee and Kumar surveyed anomaly-detection techniques and described statistical approaches that model normal behaviour and flag observations that deviate from it [5]. A z-score against a trailing window is the simplest member of that family. It needs no labelled failures, is cheap to compute and is explainable to a maintenance engineer, but it is sensitive to window length and to non-stationary behaviour. The IIA adopts this approach, together with registered engineering ranges, because explainability and the absence of labelled data per tenant were stronger requirements than detection sophistication (Section 4.10).

2.3 PUBLIC DATA AND FAILURE MODES

Matzka published the AI4I 2020 predictive-maintenance data set, a 10,000-row synthetic data set that reflects real industrial failure modes, with five failure types: tool wear (TWF), heat dissipation (HDF), power (PWF), overstrain (OSF) and random failures (RNF) [3]. The project's repository holds this data set (10,000 rows; verified by a row count of the raw CSV)

and reuses its published failure-mode thresholds in the demo tenant's rule pack, so that the knowledge base and detection rules agree.

2.4 DISTRIBUTED DATA PROCESSING

Apache Spark provides a unified engine for batch data processing with DataFrame operations and window functions [4]. For per-machine rolling statistics over time-ordered readings, window functions partitioned by machine are a natural fit. The project runs Spark in local mode inside the worker, which exercises the same DataFrame code that would run on a cluster, while the project's own documentation states that the current data volume does not require one.

2.5 RETRIEVAL-AUGMENTED GENERATION AND LEXICAL RETRIEVAL

Lewis et al. introduced retrieval-augmented generation, in which a generator is conditioned on passages retrieved from an external corpus rather than relying on parametric knowledge alone [1]. Lexical ranking with BM25 remains a strong, dependency-free baseline [2]; dense retrieval with embeddings handles paraphrase better. The IIA implements both behind one retriever contract (Section 5.10): a BM25 index for offline use and testing, and a Vertex AI embedding plus Pinecone path for semantic search. The project's own notes record that BM25 scores do not support an absolute relevance cutoff, so a term-coverage floor is used instead.

2.6 LLM AGENTS, TOOL USE AND HALLUCINATION

Yao et al. proposed interleaving reasoning and acting so that a language model can call tools and use the observations [6], and Schick et al. showed that models can learn to decide when to call external tools [7]. Ji et al. surveyed hallucination in natural-language generation and its risks [8]. The IIA draws a deliberate boundary: tool calling is allowed only for two supplementary evidence tools, and the model has no tool that can influence risk (Section 5.9). Synthesised prose is also checked against the evidence before it is shown.

2.7 MULTI-TENANT ISOLATION AND ACCESS CONTROL

PostgreSQL row-level security attaches policies to tables so that each query sees only rows permitted for the current setting [11]. When the policy keys on a transaction-local setting, isolation holds even if an application query omits a tenant filter. For credentials, argon2 is a memory-hard password-hashing function designed to resist brute-force attacks [9], and JSON Web Tokens provide a compact signed claim format [10]. PostgreSQL's `SKIP LOCKED` clause

lets several workers claim distinct rows from a queue table without blocking one another [12]. All of these are used in the IIA and are described in Chapters 4 and 5.

2.8 COMPARISON OF APPROACHES

Table 2.1: Comparison of conventional approaches with the proposed system

Aspect	Manual / spreadsheet analysis	Generic LLM chatbot	Proposed IIA
Evidence collection	Manual, slow	Not performed unless the user pastes data	Automatic: sensor trend, anomalies, history, documents
Risk level	Analyst judgement	Free text, not auditable	Deterministic rule on sustained anomalous-reading rate
Numbers in the answer	As computed by the analyst	May be invented [8]	Computed in SQL/Python; model prose is rejected if it contains numbers absent from evidence
Documentation	Searched by hand	Model's own knowledge	Retrieved passages with citations; empty result reported as such
Multiple customers	Separate files	Not applicable	Tenant isolation by credential, query filter and Postgres RLS
Customer data formats	Manual clean-up	Not applicable	Declarative mapping with unit conversion and per-row rejection report

The comparison is qualitative and describes design properties, not measured performance. No benchmark against other products was carried out in this project.

2.9 RESEARCH AND IMPLEMENTATION GAP

The surveyed literature largely addresses either detection algorithms or LLM techniques in isolation. What is less often described is the engineering that lets the two be combined safely in a service that several customers share: bounded model autonomy, verifiable output, tenant isolation that fails closed, and ingestion that tolerates customer-specific data. The IIA is an implementation study of that integration. It does not claim a new detection algorithm or a new retrieval method.

CHAPTER 03
MOTIVATION, PROBLEM STATEMENT AND OBJECTIVES

3.1 MOTIVATION

- Maintenance questions need evidence from several sources at once (sensor trends, anomaly events, maintenance records, documents), and gathering it manually delays decisions.
- A model that can invent a risk level or a statistic is not acceptable for operational decisions, so the computation of facts must be separated from the language that explains them.
- A platform offered to several companies must make cross-tenant data access impossible by construction, not merely unlikely.
- Each company exports data with its own column names, units and sensor sets, so ingestion must be configurable and must report bad rows rather than silently dropping or crashing.

3.2 PROBLEM STATEMENT

To design and implement a multi-tenant platform that ingests heterogeneous machine sensor data, detects abnormal behaviour, and answers natural-language operational questions with an evidence-backed report whose facts, risk level and rankings are computed deterministically, while preventing any tenant from accessing another tenant's data.

Inputs

- Tenant sensor CSV files (wide or long layout) with a mapping of columns, units, normal ranges and thresholds.
- Maintenance records and maintenance documentation (markdown knowledge base).
- A user question and an optional machine identifier.

Outputs

- Hourly health summaries, anomaly events, and a fleet-level risk ranking.
- A structured investigation report: risk level, root-cause candidates (labelled as possible), supporting evidence, recommendation and citations.
- A job record per ingest with counts and per-row rejection reasons.

3.3 EXISTING SYSTEM AND ITS LIMITATIONS

The project's own change history documents its starting point, which serves as the existing system for this report. Migration `0001_baseline_single_tenant` and the production-readiness log describe an initial single-tenant pipeline limited to AI4I-style columns

(temperature, rotational speed, torque and tool wear): the schema had no tenant column, queries were not tenant-filtered, thresholds were fixed constants, the pipeline was batch only, and ingestion required operator-run command-line scripts. The investigation agent existed, but its risk rule was found to be saturated (every machine of the demo fleet returned MEDIUM) and an LLM planner could skip the evidence that risk depends on. These limitations motivated the later phases.

3.4 PROPOSED SYSTEM

The proposed system, as implemented, adds tenant identity and row-level security to every company-data table; a generic long-format sensor model with a per-tenant sensor registry; an upload-to-job pipeline with a PostgreSQL queue and a background worker; a scheduled GCS pull connector; human accounts with roles; a dashboard; and a redesigned agent in which the evidence that risk depends on is always gathered deterministically.

3.5 OBJECTIVES

3.5.1 Data ingestion and detection

- Accept tenant CSV files with arbitrary sensor vocabularies using a validated mapping, converting units and reporting every rejected row with line, column and code.
- Make ingestion idempotent so that re-running an upload or retrying a failed job never duplicates data.
- Detect anomalies with explainable, per-tenant rules (rolling z-score and registered ranges) using PySpark.

3.5.2 Agent and retrieval

- Compute risk level, rankings and root-cause candidates deterministically, and limit any LLM to supplementary evidence selection and narrative.
- Provide document retrieval with citations through two interchangeable back ends, with graceful degradation when cloud services are not configured.

3.5.3 Security, operations and quality

- Derive the tenant only from the credential and enforce isolation in the application and in PostgreSQL.
- Provide role-based permissions, audit logging, rate limiting and request tracing.
- Containerise the stack and verify behaviour with an automated test suite that asserts against known ground truth.

3.6 SCOPE AND CONSTRAINTS

- Only GCS exists as a pull connector; S3/MinIO is supported only as upload storage.
- Only sensor-readings CSV is accepted through the upload API; maintenance and legacy AI4I formats use command-line scripts.
- The dashboard is a Streamlit application, not a production front end; a page refresh signs the user out.
- Data are synthetic or public; risk thresholds are calibrated on synthetic data.
- The live LLM and Pinecone paths are opt-in and are not exercised by the automated tests.

3.7 FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

Table 3.1: Functional requirements (derived from the implemented API and modules)

ID	Requirement	Realised by
FR1	Authenticate users by tenant, email and password and issue a short-lived token	`POST /auth/login`, `app/core/security.py`
FR2	Authorise each route by role; refuse undeclared routes	`app/core/permissions.py`, `enforce_route_permission`
FR3	List machines, health, anomalies, sensor trends, raw readings and bucketed series for the caller's tenant	`app/api/routes.py`, `app/api/dashboard.py`
FR4	Rank machines for inspection and summarise fleet risk	`/machines/rank-for-inspection`, `/fleet/summary`
FR5	Upload a CSV, preview it, dry-run and confirm a mapping, and queue a job	`app/api/uploads.py`
FR6	Process jobs in the background with retries, staleness recovery and per-row rejects	`app/jobs/worker.py`, `app/jobs/pipeline.py`
FR7	Pull new files from GCS on a cron schedule without double ingestion	`app/connectors/`
FR8	Investigate a machine or the fleet and store the result	`POST /investigate`, `POST /chat`, `app/agents/`
FR9	Retrieve cited maintenance documentation per tenant	`app/rag/`
FR10	Let admins edit sensor ranges and thresholds, manage users and connectors	`POST /sensors/{name}`, `/users`, `/connectors`
FR11	Record an audit trail of logins, denials and writes	`app/core/audit.py`, `audit_log`

Table 3.2: Non-functional requirements

Category	Requirement	Evidence in the project
Security	Tenant isolation must fail closed	RLS with a transaction-local setting; no context gives zero rows (`tests/test_rls.py`)
Security	No credentials in the application image or tables	Connector secrets stored by reference; DB holds only `secret_ref`; compose requires secrets from the environment
Reliability	Ingestion tolerates crashes and retries	Idempotent upserts; `jobs.locked_at` heartbeat; `iia_reap_stale_jobs()`
Observability	Trace a request through logs and audit	`X-Request-ID` middleware; node-level agent trace logging
Cost control	Bound LLM usage	Per-tenant rate limit (default 30/min), planner round cap (4), graph recursion limit (default 40), LLM timeout/retry
Portability	Run with and without cloud credentials	BM25 and deterministic report used when Vertex/Pinecone are not configured

Category	Requirement	Evidence in the project
Maintainability	Schema changes are versioned	Alembic migrations 0001-0010; test asserts migrations equal `sql/schema.sql`
Explainability	Risk and rankings are reproducible	`app/agents/risk.py` has no LLM or database access

CHAPTER 04
SYSTEM ANALYSIS AND DESIGN

4.1 OVERALL SYSTEM ARCHITECTURE

The system is organised as a browser-facing dashboard, an API process, a worker process and a PostgreSQL database. Figure 4.1 shows the components and the direction of calls. The dashboard never opens a database connection and never names a tenant; it signs in with `POST /auth/login`, keeps the JWT in its session and calls the API. The API (database role `iia_app`) and the worker (role `iia_worker`) are separate containers with separate least-privilege roles. There is no separate scheduler container: the worker's polling loop also ticks the connector scheduler.

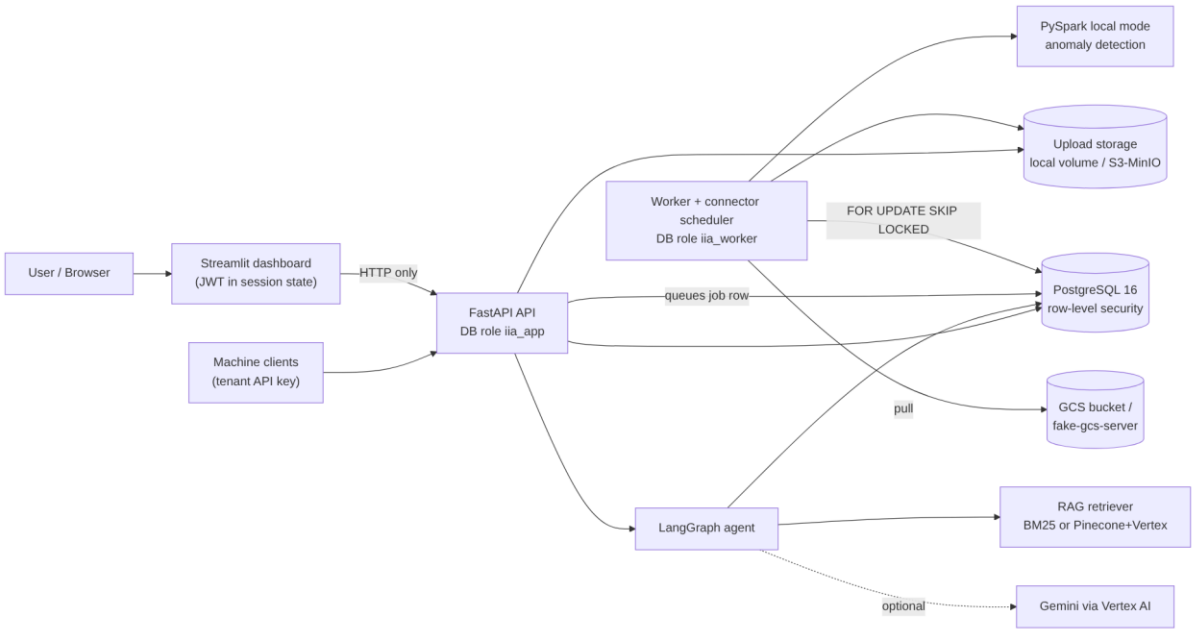


Figure 4.1: System architecture of the Industrial Intelligence Agent

4.2 MAJOR MODULES AND RESPONSIBILITIES

Table 4.1: Major modules (directory-level, from the source tree)

Module	Responsibility	Key files
`app/api`	HTTP routes, schemas, dependency injection, rate limiter, auth endpoints	`routes.py`, `dashboard.py`, `uploads.py`, `connectors.py`, `auth.py`, `dependencies.py`
`app/core`	Settings, tenancy, principal, permissions, security, audit, secrets provider, anomaly rules, logging, GenAI client	`config.py`, `permissions.py`, `security.py`, `anomaly_rules.py`
`app/database`	All SQL in one module; RLS tenant context	`queries.py`, `tenant_context.py`
`app/agents`	LangGraph graph, nodes, risk, planner, synthesis, Gemini adapter, query parser	`graph.py`, `nodes.py`, `risk.py`, `planner.py`, `llm.py`, `synthesis.py`
`app/rag`	Document loading, chunking, BM25, embeddings, Pinecone store, retriever, evaluation, manifest	`retriever.py`, `lexical.py`, `chunking.py`
`app/onboarding`	Column mapping, unit catalogue, normalisation, per-row validation report	`sensors.py`, `units.py`, `normalize.py`

Module	Responsibility	Key files
app/jobs	Queue worker and ingest pipeline	worker.py, pipeline.py
app/connectors	GCS pull connector, scheduler, validation	puller.py, scheduler.py, gcs.py
app/storage	Upload storage abstraction (local filesystem, S3-compatible)	local.py, s3.py
spark_jobs	PySpark ingestion, features, anomaly detection, long-format detection	anomaly_detection.py, feature_engineering.py, long_format.py
frontend	Streamlit app and API client	streamlit_app.py, api_client.py, views/
migrations, sql	Alembic revisions 0001-0010 and the equivalent schema.sql plus seed	versions/, schema.sql, seed.sql.gz
scripts, tests	Operator CLIs (tenants, users, onboarding, backup, partitions) and the test suite	manage_tenants.py, manage_users.py, tests/

4.3 COMPONENT INTERACTION

Three interaction patterns cover the system. (1) A **read or investigate request** travels from the dashboard to the API, passes the authentication and permission dependencies, and is answered from PostgreSQL and, for investigations, from the retriever and agent. (2) An **ingest request** stores a file through the storage provider and queues a row in `jobs`; the worker later claims it, runs the pipeline, and writes results back. (3) A **scheduled pull** is created by an admin through the API; the worker's scheduler claims due connectors and hands each new file to the same job pipeline as a manual upload. Components communicate only through HTTP (dashboard to API) and through PostgreSQL rows and the storage provider (API to worker); no message broker is used.

4.4 USE-CASE VIEW

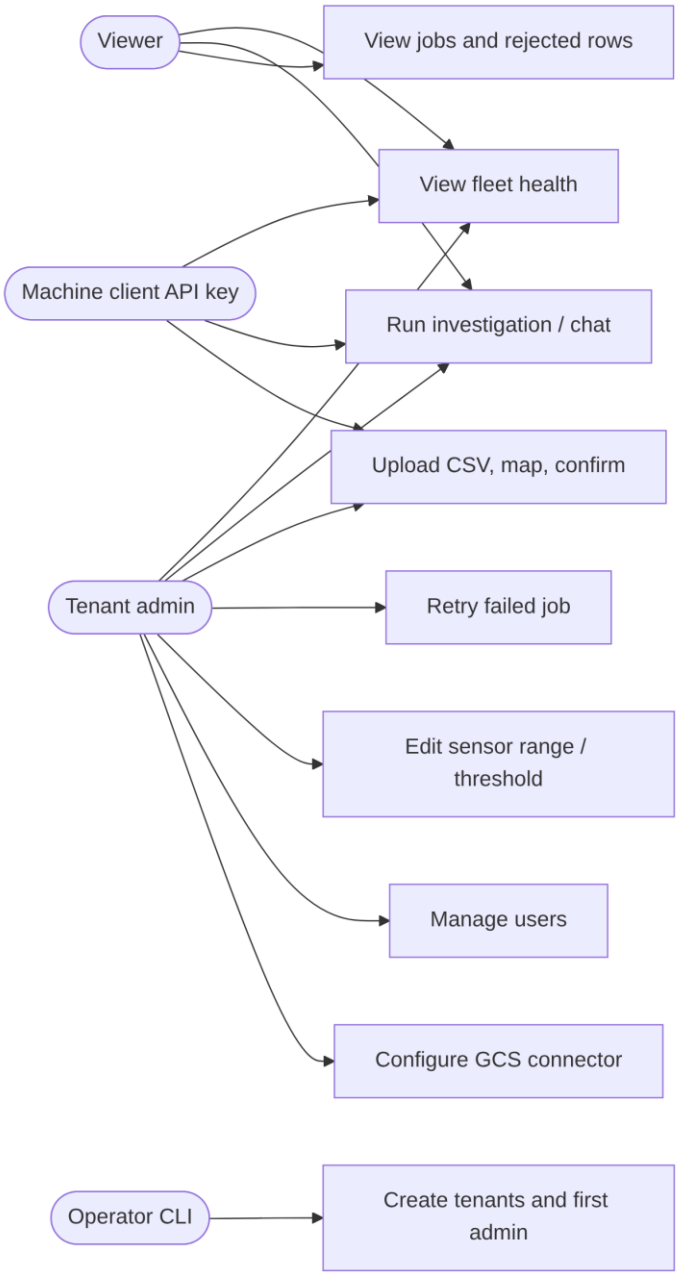


Figure 4.2: Use cases by actor

Figure 4.2 shows the actors. Viewers can read and investigate. Tenant administrators add upload, retry, sensor-registry editing, user management and connector configuration. Machine clients using a tenant API key can read, investigate and ingest, but cannot retry jobs, manage users or change configuration. The operator uses command-line scripts to create tenants and the first admin of a tenant, because there is no open sign-up.

4.5 DATA-FLOW DIAGRAM

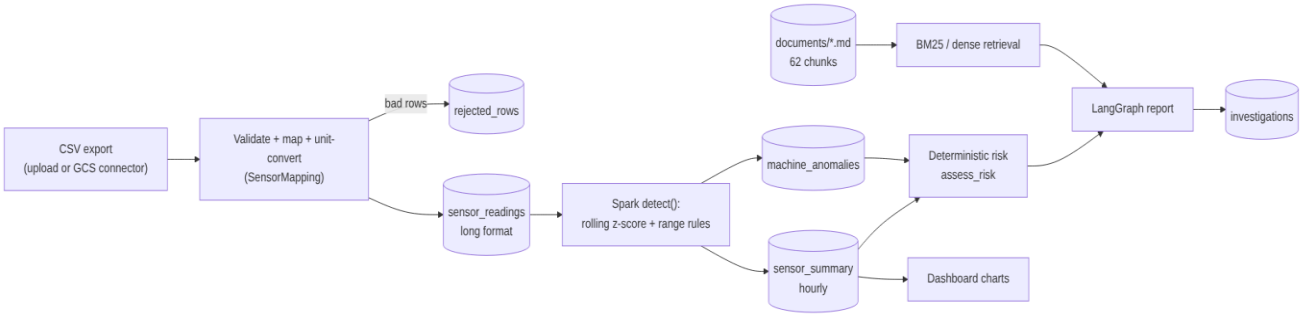


Figure 4.3: Data flow from raw export to investigation report

In Figure 4.3 a CSV export enters validation and mapping. Rejected rows are written to `rejected_rows`; accepted readings are stored in `sensor_readings` in long format. Spark detection reads the stored readings (plus baseline context) and produces hourly summaries and anomaly events. The agent reads those tables, applies the deterministic risk rule and combines the result with retrieved documentation into a report that is stored in `investigations`.

4.6 SEQUENCE DIAGRAMS

4.6.1 Investigation request

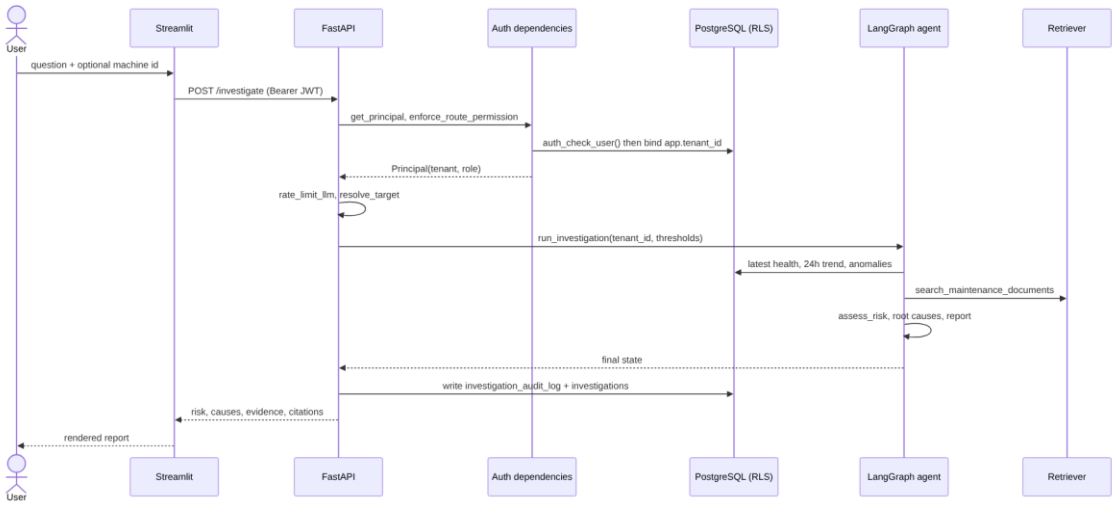


Figure 4.4: Sequence of a `POST /investigate` request

The sequence in Figure 4.4 shows that the tenant is bound to the database transaction before any query runs, that the rate limiter and target resolution precede the agent, and that the audit and investigation rows are written after the agent returns. Both writes are best effort: a failure is logged and does not turn a successful investigation into an error (see `_audit` and `_store_investigation` in `app/api/routes.py`).

4.6.2 Upload, mapping and background job

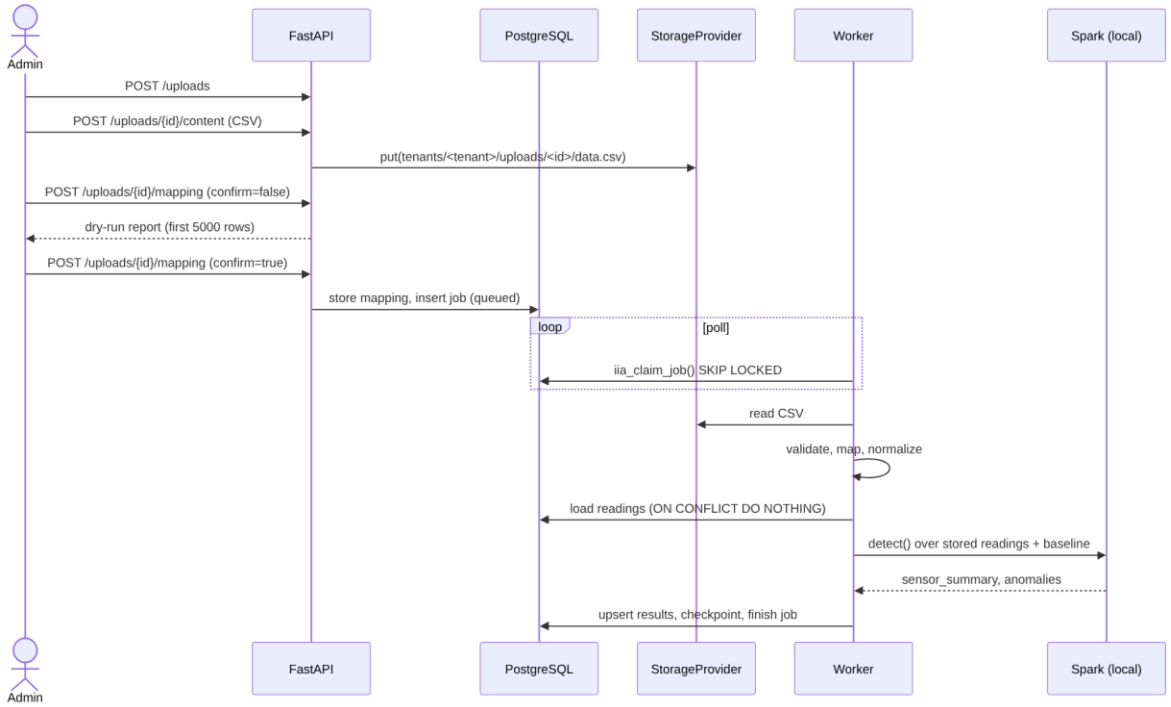


Figure 4.5: Sequence of upload, mapping confirmation and job execution

Figure 4.5 shows the two-step mapping (a dry run on a sample, then confirmation) and the worker stages. The job claim is the one cross-tenant step; it goes through a SECURITY DEFINER function that returns only the job and tenant identifiers, and every later step runs with that tenant bound.

4.7 DATABASE DESIGN

The schema in `sql/schema.sql` is the reference. Tenant-owned tables carry `tenant_id` and use composite primary and foreign keys so that a child row can only reference a parent of the same tenant (for example `sensor_summary` references `machines (tenant_id, machine_id)`). The tables `sensor_summary`, `machine_anomalies` and `sensor_readings` are declared `PARTITION BY RANGE` with a default partition; `scripts/manage_partitions.py` creates monthly partitions. `ai4i_reference` is global public reference data. Figure 4.6 shows the principal relationships and Table 4.2 summarises each table's role.

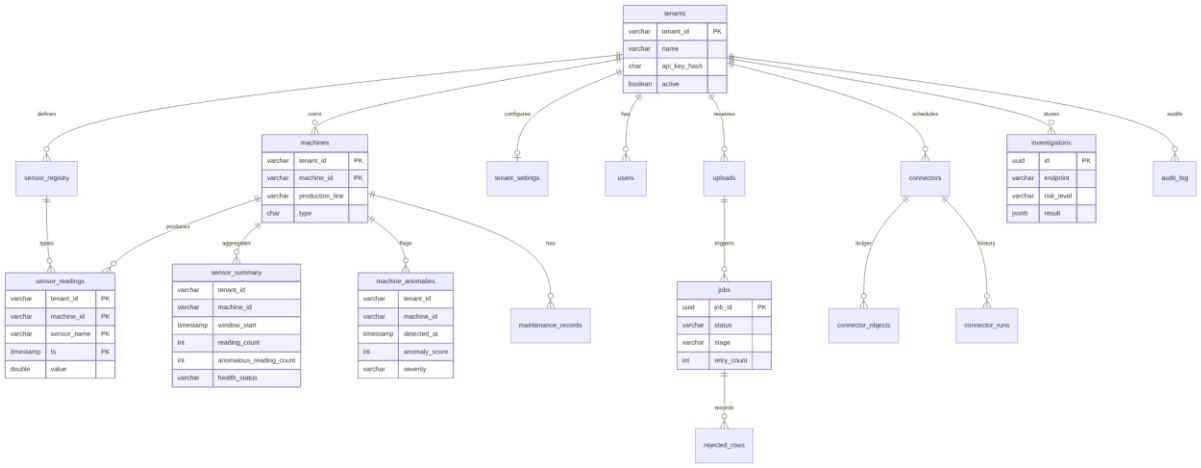


Figure 4.6: Entity-relationship diagram (principal tables and keys)

Table 4.2: Role of the main tables

Table	Role	Notable constraints / indexes
`tenants`	One row per company; stores the SHA-256 hash of its API key	`tenant_id` pattern check; `api_key_hash` unique
`machines`	Machine dimension	PK (tenant_id, machine_id); type in L/M/H or NULL
`sensor_registry`	Per-tenant sensor vocabulary, unit, normal range, z threshold, enabled flag	PK (tenant_id, sensor_name); min <= max check
`sensor_readings`	Raw long-format readings	PK (tenant, machine, sensor, ts) is the idempotency key; partitioned by `ts`
`sensor_summary`	Hourly aggregate: counts, AI4I averages, `health_status`	UNIQUE (tenant, machine, window_start); `health_status` in HEALTHY/WATCH/AT_RISK
`machine_anomalies`	Per-reading anomaly events with score, severity and reasons	UNIQUE (tenant, machine, detected_at); severity MEDIUM/HIGH
`maintenance_records`	Maintenance history	Index (tenant, machine, event_date)
`tenant_settings`	JSONB calibration: risk thresholds, RAG floor, machine-id scheme, anomaly settings	PK tenant_id
`uploads`, `jobs`, `rejected_rows`	Uploaded files, the job queue and per-row rejection reasons	`idx_jobs_queue` partial index on queued jobs; job status queued/running/succeeded/failed
`users`	Human accounts	UNIQUE (tenant, email); email lower-case check; role admin/viewer
`audit_log`, `investigation_audit_log`	General audit trail; per-investigation audit	Tenant/time indexes
`investigations`	Stored `/investigate` and `/chat` results (JSONB)	Indexes (tenant, created_at DESC) and (tenant, machine, created_at DESC)
`connectors`, `connector_objects`, `connector_runs`	Scheduled pull configuration, file ledger and run history	UNIQUE (tenant, connector, object key, version) prevents double ingestion
`ingestion_checkpoints`	High-water mark of ingested windows per source	PK (tenant, source)

4.8 API AND SERVICE ARCHITECTURE

The API is a FastAPI application (version string 0.6.0 in `app/main.py`). Routers are included with a router-level dependency, `enforce_route_permission`, which looks the

matched route up in ROUTE_PERMISSIONS. A route that is not declared is refused with 403, and a test walks every registered route to ensure none is undeclared. Table 4.3 lists the principal endpoints, taken from ROUTE_PERMISSIONS.

Table 4.3: Principal API endpoints and required permissions

Endpoint	Purpose	Permission
POST /auth/login	Tenant, email, password to JWT (public)	public
GET /machines, /machines/{id}	List fleet; one machine	data:read
GET /machines/rank-for-inspection	Deterministic ranking by anomaly volume/severity	data:read
GET /machines/{id}/health|anomalies|sensors|readings|series	Latest window; events; hourly trend; raw readings; SQL-bucketed series	data:read
GET /fleet/summary, /sensors, /units	Paged fleet risk; registry; unit catalogue	data:read
POST /investigate, POST /chat	Run the agent (single-turn); rate limited	investigate:run
GET /investigations, /investigations/{id}	Stored results (summary list; detail)	investigate:run
POST /uploads, /uploads/{id}/content, /preview	Create upload; send CSV; preview raw rows	uploads:write
POST /uploads/{id}/mapping	Dry-run or confirm a mapping (queues the job)	mapping:write
GET /uploads, /jobs, /jobs/{id}, /jobs/{id}/rejected-rows	Read uploads, jobs, rejects	uploads:read
POST /jobs/{id}/retry	Retry a failed job	jobs:retry
POST /sensors/{name}	Edit a sensor's range, threshold, enabled flag	registry:write
/connectors (create, get, update, enable, disable, delete, test, run, runs)	Scheduled GCS pull management	connectors:read / connectors:write
GET/POST /users, /users/{id}/role, /users/{id}/active	User management	users:manage
GET /, /healthz, /readyz	Landing, liveness, readiness (SELECT 1 FROM machines LIMIT 0)	public

Table 4.4: Role matrix (from `ROLE_MATRIX`)

Capability	admin	viewer	api_key
Read data, investigate, list uploads and jobs	yes	yes	yes
Upload, preview, validate or confirm a mapping	yes	no	yes
Retry a job; manage users	yes	no	no
Edit sensor registry; manage connectors	yes	no	no

4.9 SECURITY AND AUTHORIZATION DESIGN

Figure 4.7 shows how a request becomes a Principal. A Bearer token takes precedence. The token is decoded with the algorithm pinned to HS256, and only the user and tenant identifiers are trusted; the role and active flag are re-read from the database through a SECURITY DEFINER function on every request, so a demotion or deactivation applies to the next call. An API key is matched first against the legacy bootstrap key (constant-time comparison) and then by SHA-256 hash through a second SECURITY DEFINER function, because the API role cannot read tenants. A key that matches nothing is always 401. With no credential, the request is 401 when AUTH_REQUIRED is on and acts as an anonymous

viewer of the demo tenant otherwise. `AUTH_REQUIRED` defaults to on when `API_KEY` is set or the database is not local.

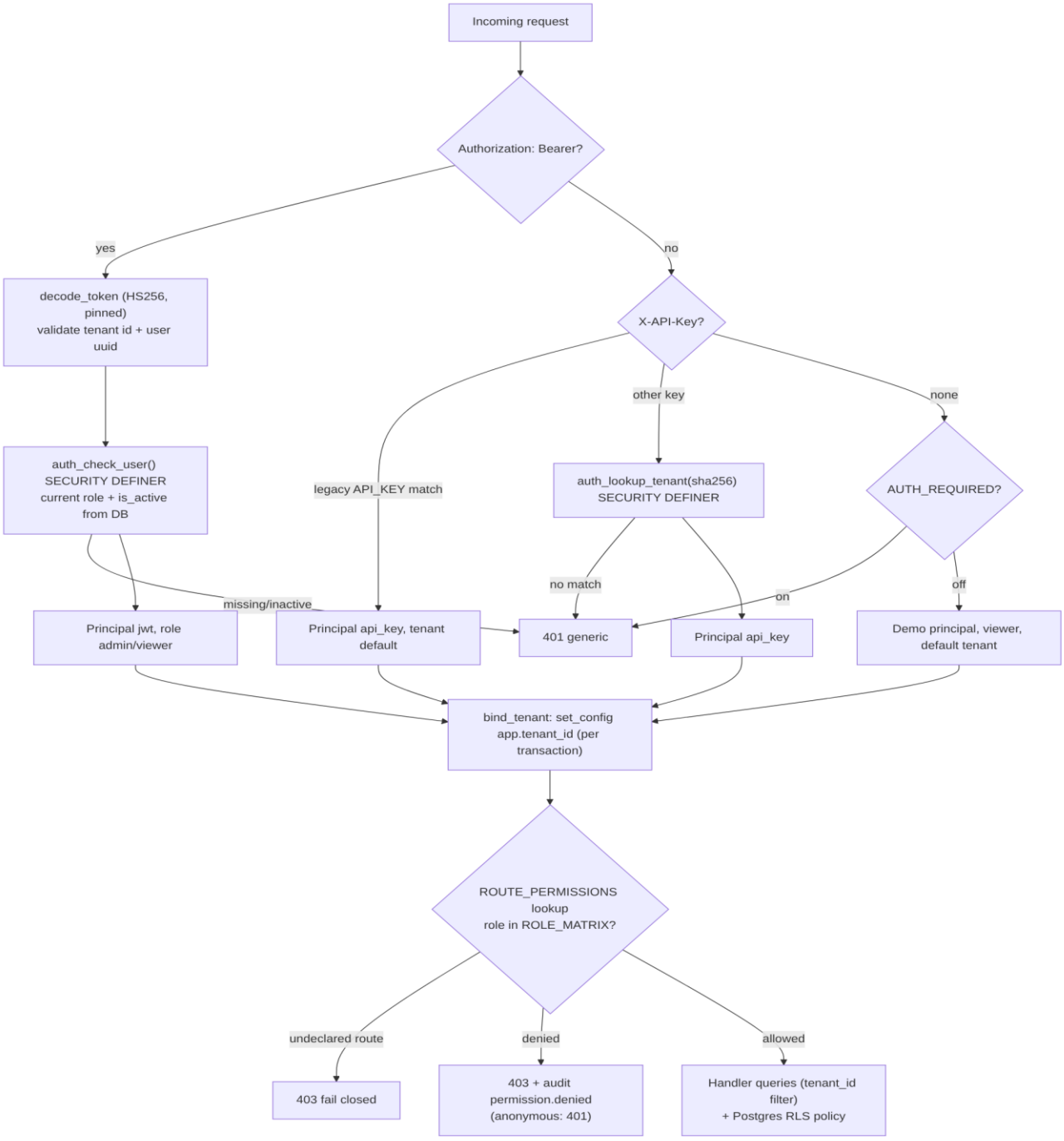


Figure 4.7: Authentication and authorization workflow

Row-level security. For each tenant table the helper `iia_enable_tenant_rls` enables and forces row-level security and creates one policy comparing `tenant_id` with `current_setting('app.tenant_id', true)`. `app/database/tenant_context.py` sets that setting with `set_config(..., true)` at the start of every transaction, so it is transaction-local and a pooled connection cannot carry a previous request's tenant. With no

context, the policy matches no rows. `jobs`, `users` and `connectors` use ENABLE without FORCE so that the owner role used by operator scripts can bypass them; this is documented in the project. At start-up the API refuses to run on a role that bypasses RLS when `RLS_REQUIRED` is on (the default for a remote database).

Other controls. Passwords use argon2id and are never returned; login performs one argon2 verification even for unknown accounts and returns one generic 401. Login is throttled to 10 attempts per minute per (tenant, email) and 30 per minute per client IP, and LLM endpoints to `RATE_LIMIT_PER_MINUTE` (default 30) per tenant, using an in-process sliding window. CORS is disabled unless an allow-list is configured, and credentials are never allowed. Connector secrets are referenced by name; the database holds only `secret_ref`, and connector error text is fixed strings. Writes, denials and logins are recorded in `audit_log` with the request identifier.

4.10 RELEVANT ALGORITHMS AND WORKFLOW LOGIC

4.10.1 Anomaly detection

For each active sensor *s* of a machine, with readings *x*_t ordered in time, the detection pass computes the trailing mean and standard deviation over the *N* readings that precede *t* (the current reading is excluded, so a value is not compared against a baseline it contributed to):

$$z_t = (x_t - \text{mean}(x_{t-N..t-1})) / \text{std}(x_{t-N..t-1}) \quad \text{if std} > 0.01, \text{otherwise } 0$$

A reading is flagged statistically when |z| exceeds the sensor's threshold (default 3.0), and flagged by range when it lies outside the registered `normal_min`/`normal_max`. The default window is 288 readings (24 hours at 5-minute sampling). A null reading never flags. The anomaly score of a reading is the number of triggered flags; severity is HIGH for a score of 3 or more and MEDIUM for 1 or 2. An hourly window is AT_RISK when its maximum score is at least 3, WATCH when at least 1 and otherwise HEALTHY. For the AI4I demo pack, additional deterministic failure-mode flags (HDF, PWF, OSF, TWF) use the published AI4I thresholds [3].

4.10.2 Deterministic risk assessment

Given the hourly trend windows of a machine, the sustained anomalous-reading rate is

$$\text{rate} = (\text{sum of anomalous_reading_count}) / (\text{sum of reading_count})$$

computed over the available trend windows when at least `MIN_TREND_WINDOWS` = 6 exist. The level is HIGH if the latest status is AT_RISK, or a HIGH-severity anomaly lies inside

the trend window, or rate >= 0.25; MEDIUM if rate >= 0.05; LOW otherwise. With fewer than six windows a conservative fallback applies (AT_RISK or a HIGH anomaly gives HIGH; WATCH or a MEDIUM anomaly gives MEDIUM). If there is no snapshot, no trend and no anomaly at all, the level is UNKNOWN rather than LOW, because LOW would claim that the evidence was checked. The two cutoffs are per-tenant tunable through `tenant_settings`.

4.10.3 Agent workflow

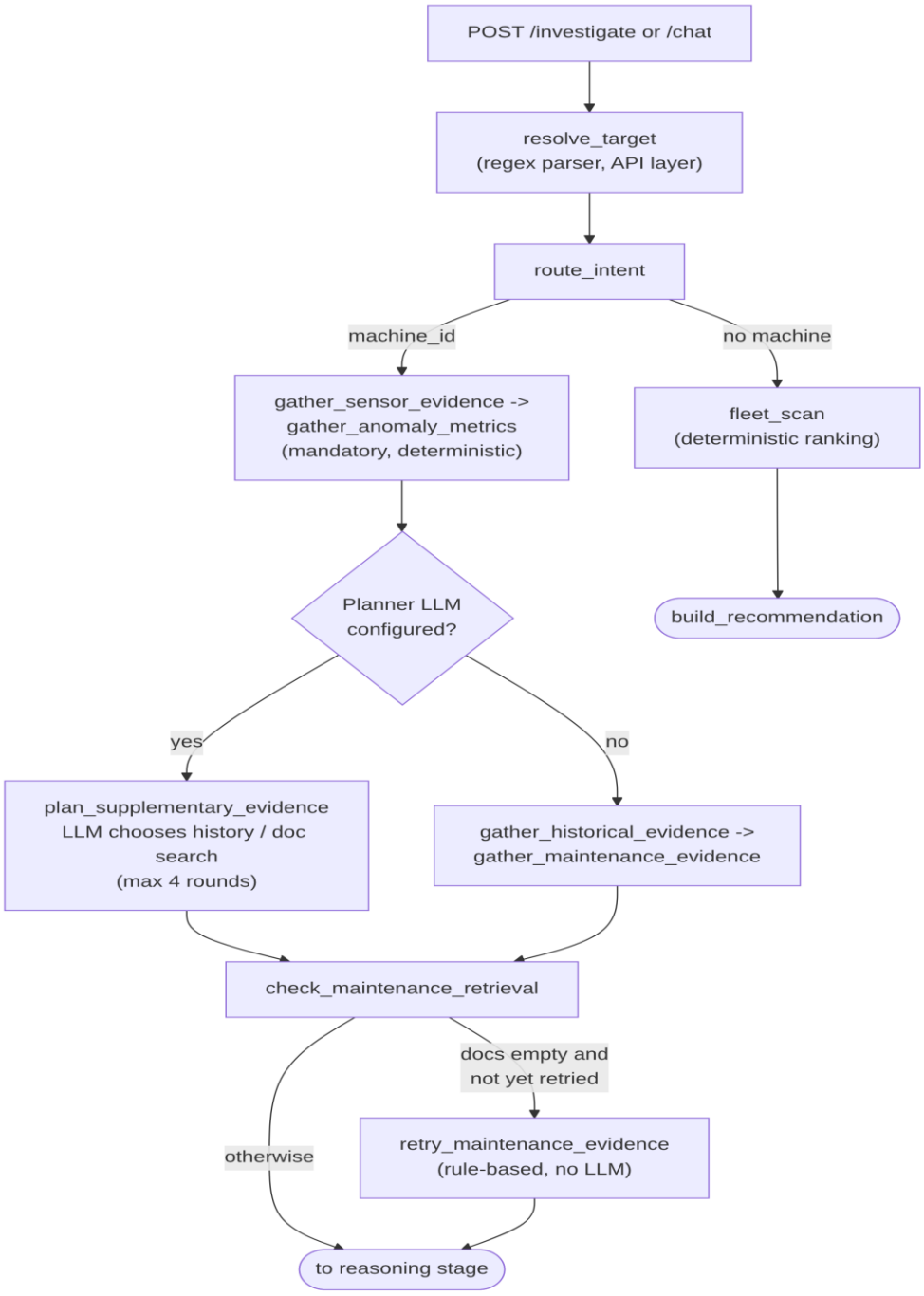


Figure 4.8: Agent workflow, stage 1: target resolution and evidence gathering

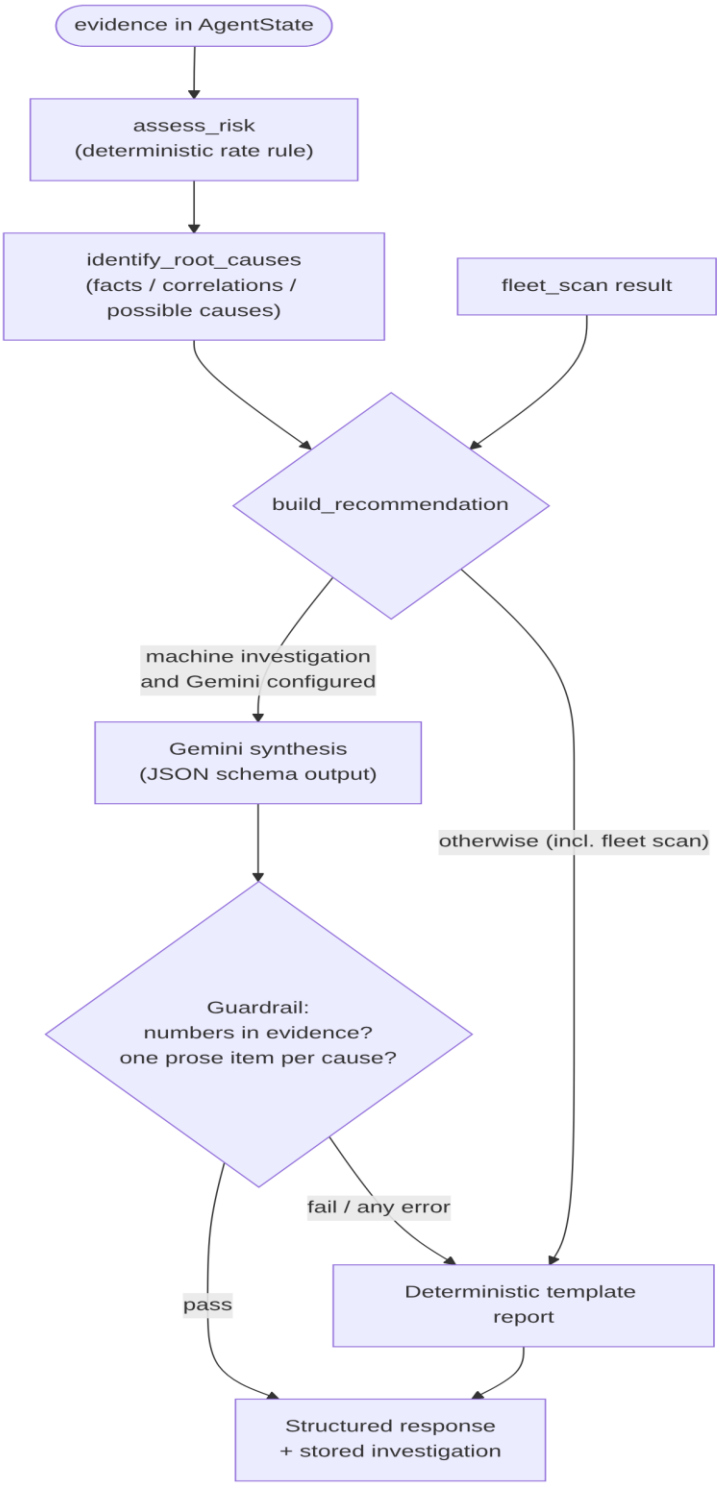


Figure 4.9: Agent workflow, stage 2: reasoning and report construction

Figures 4.8 and 4.9 show the request-processing workflow. Before the graph runs, the API resolves the target machine from the question and the optional `machine_id` using a regular-expression parser: an unknown machine named in the text is a 404, several machines or a conflicting `machine_id` is a 422, and no machine means a fleet scan. The parser is deliberately not an LLM call, since which machine a question names is a fact about the text.

4.10.4 Queue claim and retry

The worker obtains work through `iia_claim_job()`, a single `UPDATE ... WHERE job_id = (SELECT ... FOR UPDATE SKIP LOCKED LIMIT 1)`, so concurrent workers never claim the same row. A permanent error fails the job immediately; any other error re-queues it with delay min(300, base x 2^retry_count) seconds until `max_retries` (default 3) is exhausted. A job that stays `running` with a stale `locked_at` heartbeat is re-queued or failed by `iia_reap_stale_jobs()`.

CHAPTER 05
METHODOLOGY AND IMPLEMENTATION

5.1 DEVELOPMENT METHODOLOGY

The repository records an incremental, phase-based development process. Change logs `CHANGES_PHASE1.md` to `CHANGES_PHASE6.md` and `PRODUCTION_READINESS_CHANGES.md` describe successive phases (generic sensors and per-tenant rules; row-level security; uploads and the background worker; users and roles; the GCS connector; the dashboard and demo stack), each with a list of what changed, what was verified, what was not verified and the decisions taken. Three working practices are visible in these records and in the tests: (1) design decisions are recorded with their trade-offs; (2) behaviour is asserted against known ground truth, namely six failure scenarios injected into the synthetic generator, rather than generic sanity checks; and (3) a schema change requires both an Alembic revision and a matching `sql/schema.sql`, which a test compares. The project was developed on Windows with a Python 3.12 virtual environment, Java 17 for Spark and PostgreSQL (as shown in the author's setup notes); the Phase 6 verification log records a Linux run with Python 3.13 and Java 21, and the container images and CI use Python 3.12 and Java 17.

5.2 TECHNOLOGY STACK AND JUSTIFICATION

Table 5.1: Technology stack (versions from `requirements*.txt`, Dockerfiles and compose)

Layer	Technology	Why it is appropriate here
API	FastAPI 0.115.12, Uvicorn 0.34.0, Pydantic	Typed request/response models, dependency injection used for authentication and tenant binding, automatic OpenAPI docs
Database	PostgreSQL 16 (`postgres:16-alpine`), SQLAlchemy 2.0.54, psycopg2-binary 2.9.13, Alembic	Relational integrity, row-level security, partitioning, `SKIP LOCKED` queue, versioned migrations
Batch processing	PySpark 3.5.3 (local mode), pandas 3.0.2, JRE 17 in the worker image	Window functions for per-machine rolling statistics; same code scales to a cluster
Agent	LangGraph 1.2.11, langchain-core 1.6.3 (cloud extra)	Explicit state, nodes and conditional edges make the agent auditable
LLM (optional)	google-genai 2.24.0 on Vertex AI; default model identifier `gemini-3.6-flash` set in `app/core/config.py`	Structured JSON output and native function calling; credentials via Application Default Credentials, no key in code
Retrieval	BM25 (own implementation), Pinecone 10.0.0, Vertex `gemini-embedding-001` (768 dimensions), langchain-text-splitters 1.1.2	Offline default plus semantic option behind one contract
Security	argon2-cffi 25.1.0, PyJWT 2.15.1	Memory-hard password hashing; signed tokens with pinned algorithm
Connectors	google-cloud-storage 3.16.0, cronsim 2.7, fake-gcs-server (compose)	GCS listing and download; cron parsing; credential-free local emulator
Dashboard	Streamlit 1.64.0, Altair (bundled)	Rapid API-only dashboard with role-based navigation
Packaging and CI	Docker, Docker Compose, GitHub Actions, pytest 8.3.3	Reproducible stack; CI runs Alembic then the test suite

The model identifier and the Vertex and Pinecone paths are configuration defaults in the code. They were not exercised live in this project's automated tests, and the model name should be confirmed in the Vertex model catalogue before any live demonstration (the compose file carries the same caution).

5.3 ENVIRONMENT AND CONFIGURATION

Settings are read from environment variables in `app/core/config.py`, `app/core/ingest_settings.py` and `app/connectors/settings.py`; `.env.example` lists them. Docker Compose refuses to start unless the required secrets are set (`POSTGRES_PASSWORD`, `APP_DB_PASSWORD`, `WORKER_DB_PASSWORD`, `API_KEY`, `JWT_SECRET`, `JWT_TTL_MINUTES`, `DEMO_ADMIN_PASSWORD`, `DEMO_VIEWER_PASSWORD`); no default credentials exist in the compose file. Table 5.2 groups the variables by purpose without values.

Table 5.2: Configuration groups (names only; values deliberately omitted)

Group	Variables
Database and roles	`DATABASE_URL` (owner, scripts/migrations), `APP_DATABASE_URL` (API role), `WORKER_DATABASE_URL`, `APP_DB_PASSWORD`, `WORKER_DB_PASSWORD`, `RLS_REQUIRED`
API and authentication	`API_KEY`, `AUTH_REQUIRED`, `JWT_SECRET`, `JWT_TTL_MINUTES`, `CORS_ALLOW_ORIGINS`, `RATE_LIMIT_PER_MINUTE`, `UPLOAD_RATE_LIMIT_PER_MINUTE`
LLM and agent	`GOOGLE_CLOUD_PROJECT`, `GEMINI_MODEL`, `PLANNER_MODEL`, `GEMINI_LOCATION`, `GEMINI_SYNTHESIS_ENABLED`, `AGENTIC_ROUTING_ENABLED`, `LLM_TIMEOUT_SECONDS`, `LLM_MAX_ATTEMPTS`, `GRAPH_RECURSION_LIMIT`
Retrieval	`RAG_BACKEND` (auto/local/pinecone), `RAG_CHUNK_SIZE` (900), `RAG_CHUNK_OVERLAP` (150), `RAG_TOP_K`, `RAG_MIN_TERM_COVERAGE_LEXICAL` (0.3), `RAG_MIN_SCORE_DENSE` (0.75), `PINECONE_*`, `VERTEX_EMBEDDING_MODEL`, `EMBEDDING_DIMENSION`
Uploads and jobs	`STORAGE_BACKEND`, `STORAGE_LOCAL_DIR`, `UPLOAD_MAX_BYTES` (52,428,800 in compose), `JOB_MAX_RETRIES` (3), `JOB_BACKOFF_BASE_SECONDS` (5), `JOB_STALE_SECONDS` (300), `WORKER_POLL_SECONDS` (2)
Connectors	`STORAGE_EMULATOR_HOST`, `CONNECTOR_SECRETS_DIR`, `CONNECTOR_TICK_SECONDS` (30), `CONNECTOR_MIN_AGE_SECONDS` (60), `CONNECTOR_MAX_OBJECTS_PER_RUN` (25), `CONNECTOR_MIN_INTERVAL_SECONDS` (300), `CONNECTOR_OVERLAP_SECONDS`, `CONNECTOR_LEASE_SECONDS`
Frontend and demo	`API_BASE_URL`, `DEMO_ADMIN_PASSWORD`, `DEMO_VIEWER_PASSWORD`

5.4 BACKEND IMPLEMENTATION

`app/main.py` creates the SQLAlchemy engine once in the FastAPI lifespan, installs the tenant-context listener, checks that the database role does not bypass RLS, and registers the routers under the permission dependency. A middleware assigns or reuses an `X-Request-ID` held in a context variable, which is added to every log line and stored in audit rows. All SQL is in `app/database/queries.py` (1,419 lines); every function takes `tenant_id` as a required keyword-only argument, and

tests/test_tenant_isolation.py::test_every_tenant_table_query_is_tenant_filtered statically checks that tenant-table queries filter on it. Routes are thin: they validate input, call a query or the agent, and map empty results to the right HTTP status. Failures to reach the database are mapped to a generic 503 and the driver error is logged server-side only, so infrastructure details are not leaked.

5.5 DATA PIPELINE AND ANOMALY DETECTION (PYSPARK)

spark_jobs/ingestion.py creates a local-mode Spark session and loads and validates the raw files; preprocessing.py cleans them; feature_engineering.py adds derived features, the AI4I failure-mode flags and the trailing rolling statistics; anomaly_detection.py adds z-score and range flags, the risk score and severity, and builds sensor_summary and machine_anomalies; long_format.py pivots long-format readings to wide form and calls the same detect() for tenant uploads. The result is that an upload processed by the worker and the offline pipeline run directly produce the same output; a test (test_job_output_equals_the_spark_pipeline_run_directly) asserts this. The synthetic generator (scripts/generate_synthetic_data.py, seed 42) produces 18 machines on three lines over 60 days at 5-minute intervals, which is 311,040 operational rows (verified by counting the raw CSV), with six injected scenarios: M-01 TWF, M-02 HDF, M-03 PWF, M-04 OSF (left unresolved), M-05 RNF (a one-off blip) and M-06 recurring TWF.

5.6 TENANT ONBOARDING AND MAPPING

app/onboarding/sensors.py defines the mapping format (wide or long layout; machine and timestamp columns; optional timestamp format, production line and machine type; and a dictionary of sensors with source column, unit, optional target unit, normal range, z threshold and enabled flag). Units must exist in the catalogue in units.py and a conversion must stay within one physical dimension, otherwise the mapping is rejected with every problem listed. normalize_sensor_readings never raises on bad data: each problem becomes a report row with a stable code (missing_machine_id, bad_timestamp, non_numeric, non_finite, duplicate_row, conflicting_duplicate, unmapped_type and others), bad readings are dropped, and good ones are returned. Timestamps with an offset are converted to UTC; those without are taken as UTC.

5.7 DATABASE IMPLEMENTATION

Ten Alembic revisions build the schema: 0001 baseline single-tenant, 0002 multi-tenancy, 0003 tenant settings and audit, 0004 partitioning, 0005 generic sensor model, 0006 row-level security, 0007 uploads and jobs, 0008 users and audit, 0009 connectors, 0010 investigations

and registry write. `tests/test_migrations.py` asserts that `alembic upgrade head` yields a schema equal to `sql/schema.sql` and that upgrading a populated pre-tenancy database keeps every row. Partitioned tables use a default partition so existing data stay valid; monthly partitions can be added with `scripts/manage_partitions.py`. Two SECURITY DEFINER functions, `auth_lookup_tenant` and `auth_lookup_user`/`auth_check_user`, are the only paths to key hashes and password hashes, and the queue functions `iia_claim_job`, `iia_reap_stale_jobs` and `iia_claim_due_connectors` are the only cross-tenant operations.

Excerpt: tenant context applied at the start of every transaction (`app/database/tenant_context.py`)

```
def _apply_on_begin(conn: Connection) -> None:
    tenant_id = conn.get_execution_options().get(_OPTION)
    if tenant_id is None:
        return
    cursor = conn.connection.cursor()
    try:
        cursor.execute("SELECT set_config(%s, %s, true)", (GUC, tenant_id))
    finally:
        cursor.close()
```

Excerpt: RLS policy created for each tenant table (`sql/schema.sql`)

```
CREATE POLICY tenant_isolation ON <table>
  USING (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''))
  WITH CHECK (tenant_id = NULLIF(current_setting('app.tenant_id', true), ''));
```

Excerpt: queue claim (`sql/schema.sql`, function `iia_claim_job`)

```
UPDATE public.jobs j
   SET status = 'running', stage = 'claimed', started_at = (NOW() AT TIME ZONE
'UTC'),
       locked_at = (NOW() AT TIME ZONE 'UTC'), finished_at = NULL
 WHERE j.job_id = (
    SELECT q.job_id FROM public.jobs q
     WHERE q.status = 'queued' AND q.run_after <= (NOW() AT TIME ZONE 'UTC')
     ORDER BY q.run_after, q.created_at
     FOR UPDATE SKIP LOCKED LIMIT 1)
RETURNING j.job_id, j.tenant_id
```

5.8 AUTHENTICATION AND AUTHORIZATION IMPLEMENTATION

`app/core/security.py` implements argon2id hashing (library defaults), a minimum password length of 12, email normalisation, and HS256 tokens with claims `sub`, `tid`, `iss`, `iat`, `exp` and `jti`; decoding requires all of them and pins the algorithm. `app/api/dependencies.py` contains `get_principal`, `get_tenant`, `enforce_route_permission` and `rate_limit_llm`. Write permissions are audited whether they succeed or fail. User management (`app/api/auth.py`) is confined to the caller's tenant, has no open sign-up, and refuses to demote or deactivate the last active admin, using a row lock on the tenant's active admins to serialise concurrent changes.

5.9 AI AND AGENT IMPLEMENTATION

5.9.1 What is conventional logic and what is model-driven

Table 5.3: Conventional logic versus AI-driven behaviour

Step	Mechanism	LLM involved?
Which machine a question is about	Regex / set membership in `query_parser.py`; per-tenant machine-id scheme	No
Sensor and anomaly evidence	SQL via `tools.py`, mandatory nodes	No
Maintenance history and document search	Either two fixed calls, or a Gemini tool-calling planner choosing whether and how to search	Optional (planner)
Retry of an empty document search	Rule-based reformulation (drop machine id, add failure-mode filter), exactly once	No
Risk level	`assess_risk` (rate rule)	No
Root-cause candidates	Matching anomaly reasons to failure modes and documents; labelled 'possible'	No
Fleet ranking and fleet report	Deterministic template	No
Narrative report for a machine	Gemini JSON output validated against a schema and the evidence, else deterministic template	Optional (synthesis)
Dense retrieval embeddings	Vertex `gemini-embedding-001` with RETRIEVAL_DOCUMENT / RETRIEVAL_QUERY task types	Optional (embedding model)

With no `GOOGLE_CLOUD_PROJECT`, none of the optional steps run and the whole agent is deterministic. When the project is configured, the planner is feature-detected and runs by default (`AGENTIC_ROUTING_ENABLED` is an explicit kill-switch), whereas synthesis stays off until `GEMINI_SYNTHESIS_ENABLED=true`. In both cases a failure degrades to the deterministic path and is recorded in the response's errors instead of raising.

5.9.2 Graph construction and tracing

`build_graph` constructs a fresh `StateGraph` per request because the database connection and retriever are request-scoped. Every node is wrapped by `_traced`, which logs the node name, machine, duration and failures. `run_investigation` invokes the compiled graph with a recursion limit from `GRAPH_RECURSION_LIMIT` (default 40); a `GraphRecursionError` is mapped to a clean 500. The planner is built only when both a machine id and a planner client exist, and it is bounded by `MAX_PLANNER_TOOL_ROUNDS` = 4, enforced from state. The planner is offered exactly two tools (historical maintenance records and document search); sensor and anomaly tools are not offered at all, which a test checks (`test_planner_is_never_offered_sensor_or_anomaly_tools`).

5.9.3 Synthesis guardrail

`GeminiSynthesizer.synthesize` requests JSON conforming to `GeminiReportSections`, validates it, and then applies

`_validate_report_against_evidence`: the number of possible-cause paragraphs must equal the number of deterministic root-cause candidates, and every substantive number in the prose (two or more digits, or a decimal, with optional percent sign; timestamp components and single digits excluded) must occur somewhere in the evidence JSON. Any violation raises, and `build_recommendation` catches every exception and falls back to the deterministic template, recording the reason. The check is a heuristic for numeric hallucination and does not verify non-numeric claims.

Excerpt: the deterministic risk rule (`app/agents/risk.py`)

```
if sensor_evidence is None and not sensor_trend and not anomaly_metrics:
    return "UNKNOWN"
status = (sensor_evidence or {}).get("health_status")
rate = _anomalous_rate(sensor_trend, thresholds)
if rate is not None:
    if (status in _AT_RISK_STATUSES
        or _has_recent_high_anomaly(anomaly_metrics, sensor_trend)
        or rate >= thresholds.sustained_high_rate):
        return "HIGH"
    if rate >= thresholds.sustained_medium_rate:
        return "MEDIUM"
    return "LOW"
```

5.10 RETRIEVAL IMPLEMENTATION

Seven synthetic markdown documents in `documents/` (four types of procedure and reference: SOP-101, SOP-102, SOP-103, MAN-200, SAF-300, TRB-400 and incident reports INC-500) declare `data_class: SYNTHETIC` in frontmatter; the loader rejects a document without it. `chunking.py` splits by Markdown headers and then recursively by size (900 characters, 150 overlap by default) to produce 62 chunks, the count verified in `document_chunks.jsonl`; each chunk's `embedding_text` is prefixed with its document id, title and section so the vector encodes provenance. `retriever.py` defines one search contract with `LexicalRetriever` (BM25, k1 = 1.5, b = 0.75, plus a term-coverage floor of 0.3) and `DenseRetriever` (Vertex embeddings and Pinecone cosine search with a 0.75 floor, an uncalibrated starting value according to the code comments). `RAG_BACKEND=auto` picks dense only when both Pinecone and Google Cloud credentials exist; if the dense path cannot be built, a warning is logged and BM25 is used. Retrievers are cached per tenant (at most 32) and each tenant has its own chunk file or Pinecone namespace, so a tenant with no documents gets an empty index. A manifest records content hashes and refuses ingestion that looks like a version regression unless forced. `evaluation.py` holds ten query-to-document relevance cases and computes hit rate and mean reciprocal rank.

5.11 BACKGROUND JOBS AND CONNECTORS

The worker (`python -m app.jobs.worker`) polls every `WORKER_POLL_SECONDS`, reaps stale jobs, ticks the connector scheduler every `CONNECTOR_TICK_SECONDS` (not in `--once` mode) and processes one job at a time. The pipeline stages are validate, map, normalize, load, detect and store, and each commits separately: readings use `INSERT ... ON CONFLICT DO NOTHING` on the primary key, summaries and anomalies are upserts, and rejected rows are replaced wholesale on each attempt, so retries create no duplicates. Detection re-reads each machine's stored readings from its first uploaded timestamp plus `rolling_window_readings` earlier readings as baseline, so two uploads give the same detection as one combined file (a test asserts this). Spark timestamps are rendered as UTC strings before `toPandas`, which fixes a time-zone shift observed on an Asia/Calcutta host.

A connector (`app/connectors/`) is configured by an admin with bucket, prefix, filename pattern, a five-field UTC cron schedule, a mapping and an optional `secret_ref`. The scheduler claims due connectors with one atomic `SKIP LOCKED` update that stamps a lease. Each new file version is downloaded, stored like a manual upload, and recorded together with its upload and job rows in one transaction, protected by the unique ledger key (tenant, connector, object key, GCS generation). The listing watermark advances only past files whose job was accepted or that were permanently rejected, so a transiently failing or very new file is seen again while one bad file never blocks the others. Configuration accepts only bucket, prefix and pattern, so a tenant cannot point the worker at an arbitrary host; without a `secret_ref` a connector can run only against the emulator.

5.12 FRONTEND IMPLEMENTATION

`frontend/streamlit_app.py` is a thin client. If no token is held it shows only the sign-in page; otherwise `navigation.py` returns the pages for the role (Fleet health, Investigations, Jobs for everyone; Upload data and Admin for admins). `api_client.py` is the only module that calls the API; it attaches the JWT, normalises errors, caches reads for 20 seconds with the token's hash in the cache key, clears the cache on writes and sign-out, and treats a 401 as sign-out. The upload view maps wide-format files with widgets and accepts a pasted JSON mapping for other cases. The API refuses a viewer's writes regardless of what the interface shows.

5.13 DOCKERIZATION AND DEPLOYMENT ARCHITECTURE

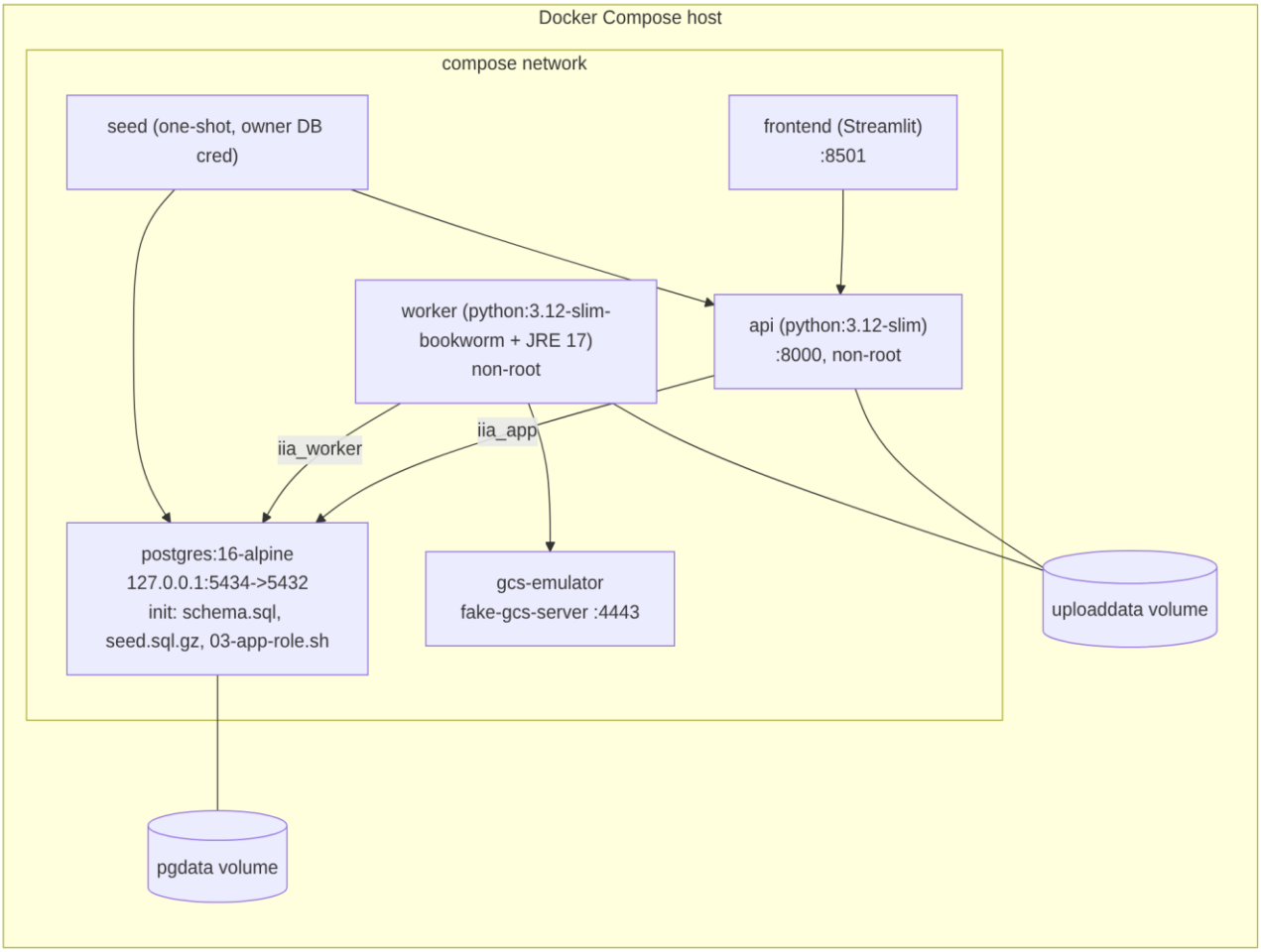


Figure 5.1: Docker Compose deployment architecture

Table 5.4: Compose services

Service	Image / build	Purpose and dependencies
`postgres`	`postgres:16-alpine`	Applies `schema.sql`, `seed.sql.gz` and `03-app-role.sh` on an empty volume; published on 127.0.0.1:5434; health check `pg_isready`
`api`	`Dockerfile` (python:3.12-slim, non-root uid 1000)	FastAPI on :8000 as `iia_app`; depends on healthy postgres; shares the upload volume
`worker`	`Dockerfile.worker` (python:3.12-slim-bookworm + OpenJDK 17 JRE)	Queue consumer and connector scheduler as `iia_worker`; `restart: unless-stopped`; can be scaled
`gcs-emulator`	`fsouza/fake-gcs-server:latest`	Demo GCS emulator on 127.0.0.1:4443 (image tag unpinned)
`seed`	`Dockerfile` (one-shot)	Runs `scripts/seed_demo.py`: Company B tenant, demo logins, sample upload; holds the owner DB credential; demo only
`frontend`	`frontend/Dockerfile`	Streamlit on :8501; calls the API only

Volumes are `pgdata` and `uploaddata` (shared by API and worker). Each image runs as a non-root user, and the API image excludes PySpark; only `spark_jobs/config.py` is copied because the anomaly-rules module imports it. Cloud dependencies are installed only when `INSTALL_CLOUD_DEPS=true`. The Phase 6 log states that `docker compose up` itself was

not verified (only `docker compose config` and the same programs as separate processes), so the container build remains to be confirmed on a machine with Docker.

5.14 END-TO-END EXECUTION WORKFLOW

1. An operator creates a tenant and its first admin with `scripts/manage_tenants.py` and `scripts/manage_users.py` (or `docker compose up` runs the demo seed).
2. The admin signs in; the dashboard stores the JWT and shows the pages permitted for the role.
3. The admin uploads a CSV, previews it, dry-runs a mapping on a sample and confirms it; the API stores the mapping and queues a job.
4. The worker claims the job, validates and normalises the file, writes rejects, loads readings, runs Spark detection and stores hourly summaries and anomalies.
5. A user opens Fleet health, which calls `/fleet/summary` and `/machines/{id}/series`; risk is computed with the same `assess_risk` rule the agent uses.
6. A user asks a question; the API resolves the target machine, the agent gathers evidence, assesses risk, builds candidates and the report, and the result is stored and shown with citations.

CHAPTER 06
TESTING, RESULTS AND DISCUSSION

Evidence policy. Every figure in this chapter is labelled by source: **[Run here]** means the test command was executed while preparing this report; **[Reported]** means the number is quoted from the project's own README or change logs and was not re-executed; **[Static]** means it was counted from the source files. No result has been estimated or invented. Items that need a database server, Docker, Java/Spark or cloud credentials were not run for this report.

6.1 TESTING METHODOLOGY

The suite uses pytest 8.3.3 with two markers declared in `pytest.ini`: `dense` (live Vertex AI and Pinecone calls, skipped without credentials) and `load` (about one million rows, excluded from the default run). Tests assert against known ground truth: the six injected failure scenarios and a second tenant ('Company B', three pumps with their own sensors) whose results are checked for correctness and for isolation from the demo tenant. Several suites run as the least-privilege database roles so that row-level security is exercised, not bypassed. The dashboard is tested with Streamlit's AppTest against the real API, and a single end-to-end test drives the whole demo flow with the real worker.

6.2 TEST ENVIRONMENT AND PREREQUISITES

The full suite needs PostgreSQL 16 loaded with `sql/schema.sql` (or `alembic upgrade head`) and `sql/seed.sql.gz`, the least-privilege roles provisioned, Java 8/11/17 or later for the Spark tests, and the generated synthetic data. CI (`.github/workflows/tests.yml`) provides these on every push with a `postgres:16-alpine` service, Python 3.12 and Java 17, applies the Alembic chain, loads the seed, runs `pytest -m "not dense and not load"`, and appends the RAG evaluation table to the job summary. A weekly or manual `load-test` job exists. The README refers to `pytest -m "not dense"`; the CI file additionally excludes `load`, which is a minor documentation difference.

6.3 TEST INVENTORY

Table 6.1: Test functions by area [Static: counted `def test_` in `tests/`; parametrised tests expand to more cases]

Area	Modules	Test functions
Agent, planner, query parser, synthesis	`test_agent`, `test_planner`, `test_query_parser`, `test_llm_synthesis`	90
Retrieval	`test_rag`, `test_rag_dense`, `test_rag_manifest`, `test_rag_tenancy`, `test_vector_store`	75

Area	Modules	Test functions
Security and tenancy	test_auth, test_rls, test_tenant_isolation, test_hardening, test_security_defaults	80
API, dashboard, end to end	test_api, test_dashboard_api, test_frontend, test_e2e_demo	49
Onboarding, ingestion, jobs, detection	test_onboarding, test_sensor_onboarding, test_uploads_pipeline, test_company_b, test_anomaly_rules, test_pipeline, test_incremental_load	83
Connectors and storage	test_connectors, test_connectors_gcs, test_connectors_migration, test_storage	25
Database, migrations, operations	test_postgres, test_migrations, test_dashboard_migration, test_backup_restore, test_load_1m	28
Total	34 test modules	**430**

6.4 TEST CASES: EXPECTED BEHAVIOUR

Table 6.2: Representative test cases (names from the suite; expected behaviour from the test names and docstrings)

Test	Verifies
test_every_tenant_table_query_is_tenant_filtered	Every tenant-table query in queries.py includes a tenant filter
test_no_context_means_zero_rows (RLS)	With no tenant bound, a tenant table returns no rows
test_cross_tenant_insert_is_rejected_and_own_insert_works (RLS)	WITH CHECK blocks writing another tenant's row
test_pooled_connection_reuse_never_leaks_a_tenant	A pooled connection does not carry a previous tenant
test_adversarial_other_tenants_machine_id_is_404_on_every_route	Tenant A's key with tenant B's machine id yields 404
test_every_registered_route_declares_a_permission	No route is undeclared in ROUTE_PERMISSIONS
test_deactivation_and_role_change_apply_to_the_next_request	Role and active flag are read from the database each request
test_failed_logins_are_indistinguishable	One generic 401 for every login failure
test_no_evidence_at_all_is_unknown_risk	Risk is UNKNOWN, not LOW, when nothing was gathered
test_lazy_planner_skipping_sensor_and_anomaly_tools_is_unknown_not_low	A planner that calls nothing cannot lower a machine's risk
test_the_same_upload_twice_changes_nothing	Ingestion is idempotent
test_a_crash_mid_job_then_retry_does_not_double_load	A retry after a crash adds no duplicates
test_two_uploads_give_the_same_detection_as_one	Rolling baselines continue across uploads
test_two_schedulers_claim_disjoint_connectors	Concurrent schedulers never run one connector twice
test_a_planted_secret_never_appears_in_logs_responses_errors_audit_or_any_table	Connector secrets do not leak
test_llm_endpoints_return_429_with_retry_after_and_reads_are_not_limited	Rate limit applies only to LLM endpoints

6.5 RESULTS

6.5.1 Tests executed while preparing this report [Run here]

A copy of the project was placed in a scratch directory, the `.env` file was removed, and seven modules that do not need a database server or Spark were executed in a clean Python 3.13 environment with `pytest -p no:cacheprovider`. Table 6.3 shows the outcome.

Table 6.3: Offline test modules executed for this report [Run here]

Module	Passed	Skipped	Failed
`test_query_parser`	44	0	0
`test_rag`	59	4	0
`test_planner`	0	1 (whole module skipped; needs `google.genai` / planner dependencies)	0
`test_llm_synthesis`	13	0	0
`test_rag_manifest`	9	0	0
`test_storage`	9	0	0
`test_security_defaults`	18	1	0
Total	**152**	**6**	**0**

The database-backed suites (RLS, tenant isolation, authentication, uploads, connectors, migrations, end to end), the Spark-dependent suites and the Docker build were **not** run for this report, because no PostgreSQL server or Spark installation was available in the preparation environment. Their results below are therefore as reported by the project.

6.5.2 Results recorded in the project documentation [Reported]

Table 6.4: Results quoted from `README.md` and `CHANGES_PHASE*.md` (not re-executed)

Item	Reported result	Source
Full suite, Phase 6 (Postgres 16, Java 21, Python 3.13, `-m "not dense and not load"`)	490 passed, 0 failed, 10 skipped (6 need `google.genai`, 4 need `pinecone`), 18 deselected	README, CHANGES_PHASE6
Suite size at Phase 1	358 passed, 0 failed, 10 skipped, 18 deselected	CHANGES_PHASE1
AI4I rule pack equals the committed Parquet output	25,920 summary rows and 28,855 anomaly rows match exactly	CHANGES_PHASE1
Company B end-to-end	Job succeeds through real worker and Spark: 7,776 readings, 3 machines, 48 anomalies; tenant B sees nothing of A	CHANGES_PHASE6
Whole stack as processes	`scripts/e2e_demo.py` printed `E2E OK`; viewer login sees PUMP-A101 and PUMP-A102 at MEDIUM, PUMP-A103 at LOW; default admin sees 18 machines	CHANGES_PHASE6
RAG evaluation (10 cases, local BM25)	Hit rate 100%, strict pass rate 100%, MRR 0.950	PRODUCTION_READINESS_CHANGES
1M-row index check	1,080,018 rows generated and queried in about 16 s; both queries used an index scan	PRODUCTION_READINESS_CHANGES

Item	Reported result	Source
Backup and restore drill	Dump restored into a scratch database; every table's row count matched	PRODUCTION_READINESS_CHANGES

The project's own text states which items were never verified: `docker compose up` from a clean clone, the roughly 2.5-million-row long-format load of the AI4I demo, the `dense` tests against real Vertex and Pinecone services, the connector against a running emulator, GitHub Actions, and any real browser session. The README's count (490 passed, 10 skipped, 18 deselected) is larger than the 430 `def test_` functions counted statically, which is consistent with parametrised tests expanding into several cases, but the difference has not been reconciled by running collection.

6.6 APPLICATION SCREENSHOTS

The uploaded project contains no screenshots (the README's Screenshots section says they were not included, and no image files exist in the archive). Rather than create images that could be mistaken for the running application, the figures below are marked placeholders. Genuine screenshots should be captured from a running instance and inserted before submission.

[PLACEHOLDER - insert genuine screenshot] Capture the Fleet health page after signing in as a tenant admin.

Figure 6.1: Fleet health page

[PLACEHOLDER - insert genuine screenshot] Capture an investigation result (for example for machine PUMP-A101 after the demo seed).

Figure 6.2: Investigations page showing a machine report with citations

[PLACEHOLDER - insert genuine screenshot] Capture the Jobs page after the sample

upload has been processed.

Figure 6.3: Jobs page showing a succeeded ingest job

6.7 DISCUSSION OF OBSERVED BEHAVIOUR

- **Risk rule.** The project's notes record that an earlier rule (any MEDIUM event gives MEDIUM) saturated, so all 18 machines returned MEDIUM, including the one whose last 24 hours were fully anomalous. The sustained-rate rule replaced it. The README example states that M-04 (unresolved overstrain) returns HIGH; this is described in the documentation and exercised by the suite, but was not re-run here.
- **Planner safety.** Removing the sensor and anomaly tools from the planner and testing a 'lazy' planner shows that model behaviour cannot reduce risk. The remaining model discretion concerns only which supplementary evidence is fetched.
- **Retrieval.** BM25 is strong on exact technical terms but weaker on paraphrase, as the evaluation notes show (two of ten cases accept rank 2 or 3). Dense retrieval is expected to help but its live behaviour is not established by the automated tests.
- **Tenant isolation.** Isolation rests on three layers (credential-derived tenant, a mandatory filter in every query, and RLS). The RLS layer is meaningful only when the API runs as a non-owner role, which the application checks at start-up.

6.8 LIMITATIONS, EDGE CASES AND KNOWN ISSUES

Table 6.5: Known limitations (from README 'Limitations' and code inspection)

Area	Limitation
Data	Synthetic and public data only; risk thresholds (5% and 25%) calibrated on the synthetic fleet, not industrial limits
Detection	Composite failure rules exist only for the AI4I pack; other tenants get per-sensor z-score and range rules; no minimum baseline, so a machine's first readings can false-flag; hourly per-sensor aggregates are stored only for AI4I columns
Agent	`/chat` is single-turn with no memory; fleet scan has no schema validation or LLM; numeric guardrail does not check non-numeric claims
Uploads and worker	Sensor-readings CSV only; S3/MinIO provider tested only against a fake client; mapping dry-run checks the first 5,000 rows; an anomaly that no longer fires after re-detection is not deleted
Accounts	No password reset, MFA, SSO, token revocation or refresh token; login throttling counts attempts and is per process; tenant ids can be probed by timing on failed login
Connectors	GCS only; secrets manager not implemented (environment or file provider); listing re-reads the whole prefix each run
Dashboard	A page refresh signs the user out; investigation history is tenant-wide and never deleted; the question parser knows only `M-NN` ids unless a tenant scheme is configured, and there is no CLI to set it
Operations	No quotas, retention, alerting or compliance workflow; rate limiter is in-process; no concurrent-user or large-upload load test
Deployment	`docker compose up` unverified in the project's own log; `gcs-emulator` uses an unpinned image tag

6.9 DISCREPANCIES BETWEEN DOCUMENTATION AND SOURCE

Table 6.6: Discrepancies found while preparing this report

Item	Finding	Treatment in this report
Test totals	README: 490 passed; static count of `def test_` is 430	Both quoted with their sources; not reconciled
Test command	README documents `pytest -m "not dense"`; CI runs `-m "not dense and not load"`	CI file taken as authoritative for CI
Python version	Phase 6 run used Python 3.13; CI and images use 3.12	Both stated
PostgreSQL host port	Compose comment says host 5433; mapping and `.env.example` use 5434	Mapping 127.0.0.1:5434 taken as authoritative
Docs describe `manage_tenants.py calibrate`	Command does not exist (README admits this)	Described only as a limitation
Archive contents	The ZIP contains `.env`, a database dump in `backups/`, test temp files and `__pycache__`	Not read into the report; see submission checklist (Appendix E)

CHAPTER 07
CONCLUSION AND FUTURE ENHANCEMENTS

7.1 SUMMARY OF THE COMPLETED WORK

The Industrial Intelligence Agent was implemented as a multi-tenant platform comprising a FastAPI service, a Streamlit dashboard, a PostgreSQL schema with row-level security and ten migrations, a PySpark detection pipeline, a PostgreSQL-queue background worker, a scheduled GCS connector, a LangGraph investigation agent with two interchangeable retrieval back ends, and a Docker Compose deployment. The repository's own records report a passing full suite at the end of Phase 6 and, in this report's preparation, 152 offline tests were executed successfully.

7.2 MAJOR TECHNICAL CONTRIBUTIONS

- A **separation of fact from language**: sensor and anomaly evidence, risk level, root-cause candidates and fleet ranking are deterministic and tested, and model output is optional, bounded and validated.
- **Fail-closed tenant isolation** built from a credential-derived tenant, mandatory query filters checked statically, composite foreign keys and forced row-level security with transaction-local context.
- **Configurable, idempotent ingestion** with a unit-aware mapping, per-row rejection reports, a PostgreSQL queue with crash recovery, and a connector ledger that prevents double ingestion.
- **Graceful degradation**: the platform runs fully offline, and every optional cloud dependency falls back to a deterministic path with the reason recorded.

7.3 EXTENT TO WHICH THE OBJECTIVES WERE ACHIEVED

Table 7.1: Objectives against evidence

Objective	Status	Basis
Mapping-driven ingestion with per-row report	Achieved (code and tests)	`app/onboarding`, `test_sensor_onboarding`, `test_uploads_pipeline`
Idempotent ingestion	Achieved (code and tests; DB tests [Reported])	Upserts and `ON CONFLICT`; retry tests
Explainable per-tenant detection with PySpark	Achieved for z-score and range rules	`spark_jobs`; composite rules only for AI4I
Deterministic risk and bounded LLM role	Achieved	`risk.py`; planner tool set; guardrail; 152 offline tests run here
Retrieval with two back ends and citations	Achieved offline; dense path unverified live	`app/rag`; `dense` tests need credentials
Tenant isolation in app and database	Achieved (DB tests [Reported])	RLS migration and suites
Roles, audit, rate limiting, tracing	Achieved	`permissions.py`, `audit.py`, `rate_limit.py`

Objective	Status	Basis
Containerised stack	Configured; `docker compose up` unverified	CHANGES_PHASE6

7.4 CURRENT LIMITATIONS

The limitations listed in Section 6.8 apply: the platform has been evaluated on synthetic and public data, calibration is not validated against a real plant, the live LLM and vector-search paths and the container build are unverified in the supplied evidence, and several production features (secrets manager, alerting, retention, password reset, MFA, quotas) are explicitly not built.

7.5 REALISTIC FUTURE ENHANCEMENTS

- Add alerting (email, chat or webhook) when a machine's risk changes level.
- Calibrate risk thresholds and detection windows on real customer data and report precision and recall of detection against labelled events.
- Add a reranking step to retrieval and extend schema validation to the fleet-scan path (both listed in the README).
- Multi-turn `/chat` with session state, if follow-up questions are required.
- Integrate a managed secrets service, token revocation, password reset and MFA.
- Add more connectors (S3 pull, SFTP, REST), and a streaming path (message broker and Spark Structured Streaming) as sketched in `docs/streaming_ingestion_design.md` once a source actually streams.
- Replace the Streamlit dashboard with a production front end and add retention and tenant-data deletion workflows.
- Capture and publish dashboard screenshots, verify `docker compose up` from a clean clone, and run the `dense` tests with real credentials.

CHAPTER 08
REFERENCES

[1] P. Lewis, E. Perez, A. Piktus, et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459-9474.

[2] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333-389, 2009.

[3] S. Matzka, "Explainable artificial intelligence for predictive maintenance applications," in *Proc. 3rd Int. Conf. on Artificial Intelligence for Industries (AI4I)*, 2020, pp. 69-74. (AI4I 2020 Predictive Maintenance Dataset, UCI Machine Learning Repository.)

[4] M. Zaharia, R. S. Xin, P. Wendell, et al., "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56-65, 2016.

[5] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, Art. 15, 2009.

[6] S. Yao, J. Zhao, D. Yu, et al., "ReAct: Synergizing reasoning and acting in language models," in *Int. Conf. on Learning Representations (ICLR)*, 2023.

[7] T. Schick, J. Dwivedi-Yu, R. Dessi, et al., "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, 2023.

[8] Z. Ji, N. Lee, R. Frieske, et al., "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, Art. 248, 2023.

[9] A. Biryukov, D. Dinu, and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symp. on Security and Privacy (EuroS&P)*, 2016, pp. 292-302.

[10] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," RFC 7519, Internet Engineering Task Force, May 2015.

[11] PostgreSQL Global Development Group, "PostgreSQL 16 documentation: Row security policies." [Online]. Available: https://www.postgresql.org/docs/16/ddl-rowsecurity.html

[12] PostgreSQL Global Development Group, "PostgreSQL 16 documentation: SELECT, the locking clause (FOR UPDATE, SKIP LOCKED)." [Online]. Available: https://www.postgresql.org/docs/16/sql-select.html

[13] LangChain, "LangGraph documentation." [Online]. Available: https://langchain-ai.github.io/langgraph/

[14] S. Ramirez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[15] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io/

[16] Docker Inc., "Docker Compose documentation." [Online]. Available: https://docs.docker.com/compose/

[17] M. Bayer, "Alembic documentation." [Online]. Available: https://alembic.sqlalchemy.org/

[18] Google Cloud, "Vertex AI documentation." [Online]. Available: https://cloud.google.com/vertex-ai/docs

[19] Pinecone Systems, "Pinecone documentation." [Online]. Available: https://docs.pinecone.io/

[20] J. Lee, B. Bagheri, and H.-A. Kao, "A cyber-physical systems architecture for Industry 4.0-based manufacturing systems," *Manufacturing Letters*, vol. 3, pp. 18-23, 2015.

[21] A. K. S. Jardine, D. Lin, and D. Banjevic, "A review on machinery diagnostics and prognostics implementing condition-based maintenance," *Mechanical Systems and Signal Processing*, vol. 20, no. 7, pp. 1483-1510, 2006.

[22] T. P. Carvalho, F. A. A. M. N. Soares, R. Vita, et al., "A systematic literature review of machine learning methods applied to predictive maintenance," *Computers & Industrial Engineering*, vol. 137, Art. 106024, 2019.

[23] The Apache Software Foundation, "Apache Spark 3.5.3 documentation." [Online]. Available: https://spark.apache.org/docs/3.5.3/

[24] pytest-dev, "pytest documentation." [Online]. Available: https://docs.pytest.org/

Project sources used as primary evidence: the submitted repository `IIA- ZIP` (source code, `README.md`, `CHANGES_PHASE1-6.md`, `PRODUCTION_READINESS_CHANGES.md`, `docs/streaming_ingestion_design.md`, `sql/schema.sql`, `docker-compose.yml`, Dockerfiles, `.github/workflows/tests.yml`).

APPENDICES

APPENDIX A: INSTALLATION AND EXECUTION

The commands below are taken from the project README and compose file; placeholders appear in angle brackets. They were not executed while preparing this report.

A.1 Docker path

```
cp .env.example .env
# set POSTGRES_PASSWORD, APP_DB_PASSWORD, WORKER_DB_PASSWORD, API_KEY,
#   JWT_SECRET   (>=   32   random   chars),   JWT_TTL_MINUTES,   DEMO_ADMIN_PASSWORD,
DEMO_VIEWER_PASSWORD
docker compose up --build
# API docs:  http://localhost:8000/docs      Dashboard: http://localhost:8501
# `docker compose down -v` clears the seeded volume for a clean re-seed
```

A.2 Local path (no Docker)

```
pip  install  -r  requirements-runtime.txt          #  +  requirements-cloud.txt  for
Pinecone/Vertex
cp .env.example .env                        # edit database URLs
alembic upgrade head
python scripts/run_pipeline.py              # PySpark; needs a local JVM
python scripts/load_postgres.py
python scripts/ingest_documents.py
uvicorn app.main:app --reload               # API on :8000
streamlit run frontend/streamlit_app.py     # UI on :8501
```

A.3 Tests

```
pytest -m "not dense and not load"   # needs Postgres (+ Java for Spark tests)
pytest -m dense                      # live Vertex AI + Pinecone, needs credentials
```

APPENDIX B: SELECTED API EXAMPLES

Representative calls documented in the README (tenant and credentials are placeholders):

```
# login
curl -s localhost:8000/auth/login -H 'content-type: application/json' \
     -d '{"tenant":"<tenant>","email":"<email>","password":"<password>"}'

# machine-specific investigation (Bearer token from login)
POST   /investigate         {"machine_id":   "M-04",   "question":   "Why   is   M-04
underperforming?"}

# upload flow
POST /uploads                       {"filename": "export.csv"}
POST /uploads/{id}/content          raw CSV body, Content-Type: text/csv
POST /uploads/{id}/mapping          {"mapping": {...}, "confirm": false}   # dry run
POST /uploads/{id}/mapping          {"mapping": {...}, "confirm": true}    # queue job
```

APPENDIX C: REPRESENTATIVE MAPPING (SECRETS REMOVED)

```
{
  "dataset_kind": "sensor_readings", "format": "wide",
  "machine_id": "AssetTag", "timestamp": "ReadTime",
  "timestamp_format": "%d/%m/%Y %H:%M",
  "sensors": {
    "discharge_pressure": {"source": "PressKPa", "unit": "kPa", "target_unit": "bar",
                           "normal_min": 1.5, "normal_max": 6.0,
                           "z_score_threshold": 3.5, "enabled": true}
```

```
  },
  "anomaly": {"z_score_threshold": 3.0, "rolling_window_readings": 288}
}
```

This is the format documented in the header of `app/onboarding/sensors.py`.

APPENDIX D: IMPLEMENTATION STATUS SUMMARY

Table 8.1: Status of major capabilities (README 'What's real vs stubbed', checked against the code tree)

Capability	Status
Tenant isolation (query filters, FK, RLS)	Implemented
Users, roles, JWT login, audit	Implemented
Upload, mapping, background job	Implemented
GCS pull connector	Implemented (GCS only; tested against in-process fake)
Dashboard	Implemented, demo-grade (Streamlit)
LLM planner and synthesis	Implemented, opt-in; not exercised live in tests
Pinecone and Vertex retrieval	Implemented, opt-in; `dense` tests need credentials
Secrets manager	Stub (environment or file provider)
Quotas, retention, alerting, compliance workflow	Not built
Password reset, SSO, MFA	Not built
Streaming ingestion	Design note only; checkpointed micro-batch exists

APPENDIX E: SUBMISSION CHECKLIST

- Fill the marked placeholders: submission month and year, internal guide, head of department, signatures and examiners. These were left blank deliberately; no approval has been assumed.
- Confirm the registration number and department details on the title page (they were copied from the reference report).
- Insert genuine screenshots in Section 6.6.
- Check the bibliography against library copies and the university's required citation style.
- Before sharing the project archive, remove `.env`, `backups/*.dump`, `.pytest-basetemp/` and `__pycache__` folders and rotate any credential that was ever stored in the archive.
- Regenerate the table of contents and lists in Word (page numbers were computed from a LibreOffice rendering and may shift slightly).